

第二章

数据仓库模型及建立过程

本章大纲

- 数据仓库设计方法概述
- 数据仓库的三级模型
- 数据仓库的概念模型
- 数据仓库的逻辑模型
- 数据仓库的物理模型
- 数据仓库的设计步骤
- 数据仓库的建设过程

1.数据仓库设计方法概述

- 数据仓库的设计与数据库的设计有以下不同：

对比内容	数据库系统设计	数据仓库系统设计
面向的处理类型	面向应用	面向分析
应用需求	比较明确	不太明确
系统设计的目标	事务处理的并发性、安全性、高效性	保证数据的四个特征和全局一致性
数据来源	业务操作员的输入	业务系统、外部数据
系统设计的方法	需求驱动	数据驱动

1.1 面向的处理类型

- 数据库系统设计
 - 面向应用来进行设计，根据具体的操作事件和操作对象(实体)来进行设计
 - 目的是建立一个操作型的数据环境
- 数据仓库设计
 - 面向分析
 - 从最基本的主题开始，不断完善已有主题，发展新主题
 - 最终建立起一个面向主题的分析型数据环境

1.2应用需求

- 数据库系统设计
 - － 面向明确的应用需求
 - － 设计人员能够清晰地了解应用的需求和数据流程
- 数据仓库设计
 - － 很难获得对用户需求的确切了解
 - － 应用人员往往是企业的中高层人员
 - － 他们自己一开始不知道想看什么，需要人引导
 - － 后期又想什么东西都看，需要给他解释

1.3 系统设计的目标

- 数据库系统设计
 - 为了进行OLTP处理
 - 通常是对一个或者一组记录的查询和修改
 - 主要为企业的特定应用服务
 - 事务处理响应时间、数据的安全性和完整性是系统的目标
- 数据仓库设计
 - 为了分析决策
 - 主要目标是保证数据的四个特征（面向主题、集成的、稳定的、时变的），建立起一个全局一致的数据环境，作为企业决策支持的基础

1.4数据来源

■ 数据库系统设计

- 数据来源主要是业务操作员的输入
- 描述如何通过操作员输入获取数据
- 描述如何将获取的数据按照OLTP的需求合理存放
- 如何使得OLTP的性能更加优化
- 如何保证事务处理的安全性

■ 数据仓库设计

- 数据来源于业务系统和外部数据
- 主要解决如何得到完整一致的数据
- 如何对数据进行转换、清洗、综合
- 如何有效提高数据分析的效率与准确性

1.5系统设计方法的不同

■ 数据库系统设计

- “**需求驱动**”，先收集需求、分析需求，再进行设计和开发
- 系统的需求在收集和分析需求阶段之后就定下来了，一旦进入构建数据库阶段，系统的需求就基本不变了
- 所以一般采用系统生命周期法(Systems Development Life Cycle , SDLC)

■ 数据仓库设计

- “**数据驱动**”，从业务系统已经存在的数据出发，获取之后对数据进行集成并检查数据的准确性
- 按照分析领域对数据及数据之间的联系重新考察，组织数据仓库中的主题
- “数据驱动”的系统设计方法的优点是可以通过了解原有数据库系统中的数据和需要建设的数据仓库中主题的数据的共同性，最大限度地利用现有系统，减少系统建设的工作量
- 对这种**需求不确定**的开发过程，设计方法有很大的不同，采用与SDLC相反的CLDS法

参考：https://blog.csdn.net/qq_44202160/article/details/123040047

2.数据仓库的三级模型

■ 两种经典的模型

– 数据库方式的模型：概念模型-逻辑模型-物理模型

- 概念模型：ER图
- 逻辑模型：主题及主题之间的联系都用关系来表示；描述了数据仓库的主题的逻辑实现，每个主题对应关系表的关系模式的定义
- 物理模型：逻辑模型在数据仓库中的实现，如数据存储结构、数据的索引策略、数据的存储策略以及存储分配优化等

– Inmon提出的三级模型

~cont. Inmon的三级模型

■ 高级模型

- 即概念模型，用E-R图表示
- 首先将现实世界抽象为概念模型，然后再用适合计算机世界的模型和语言来描述

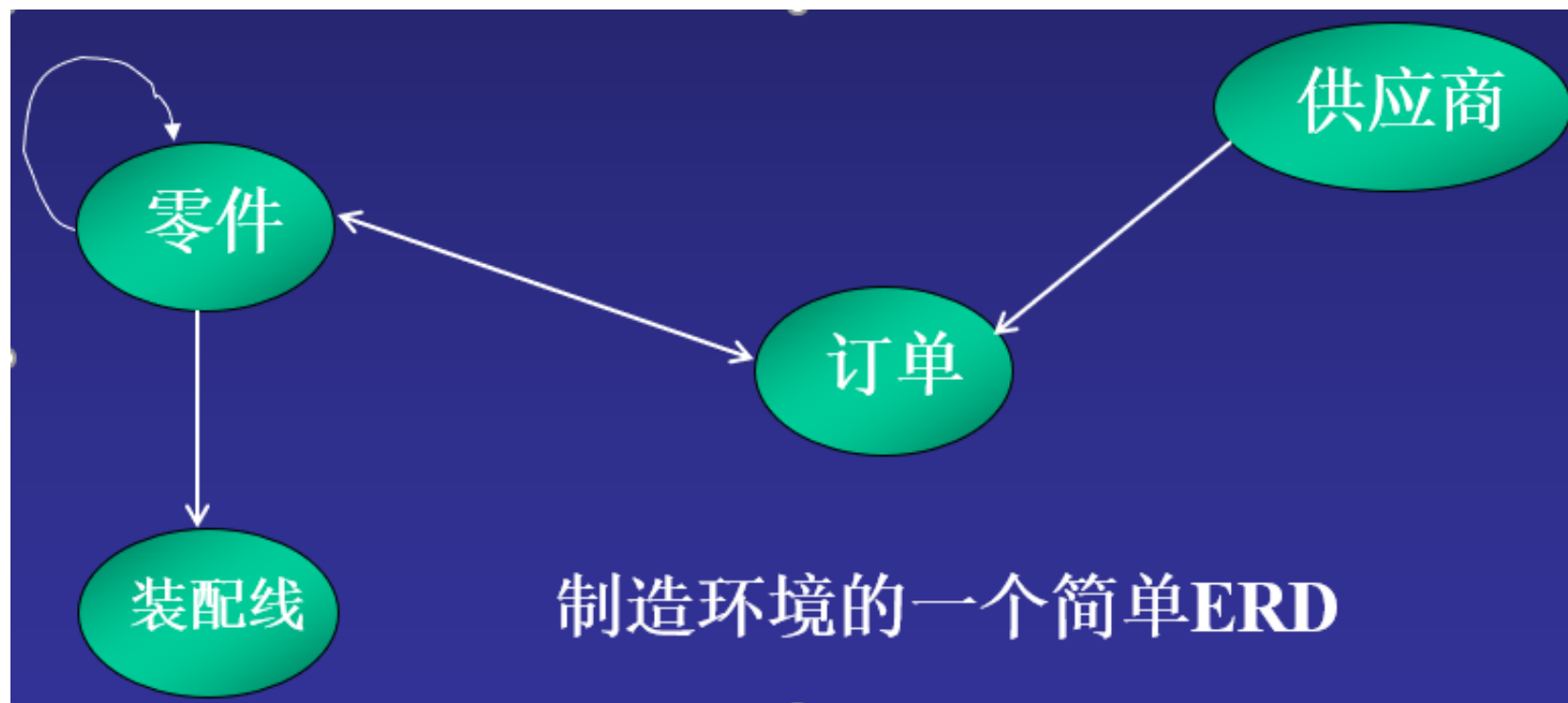
■ 中级模型

- 数据项集（Data item set, DIS）模型
- E-R图的细分
- 每个主题都与一个DIS对应
- DIS中的数据分为4个组别：基本数据组、二级数据组、连接器和类型数据组

■ 低级模型

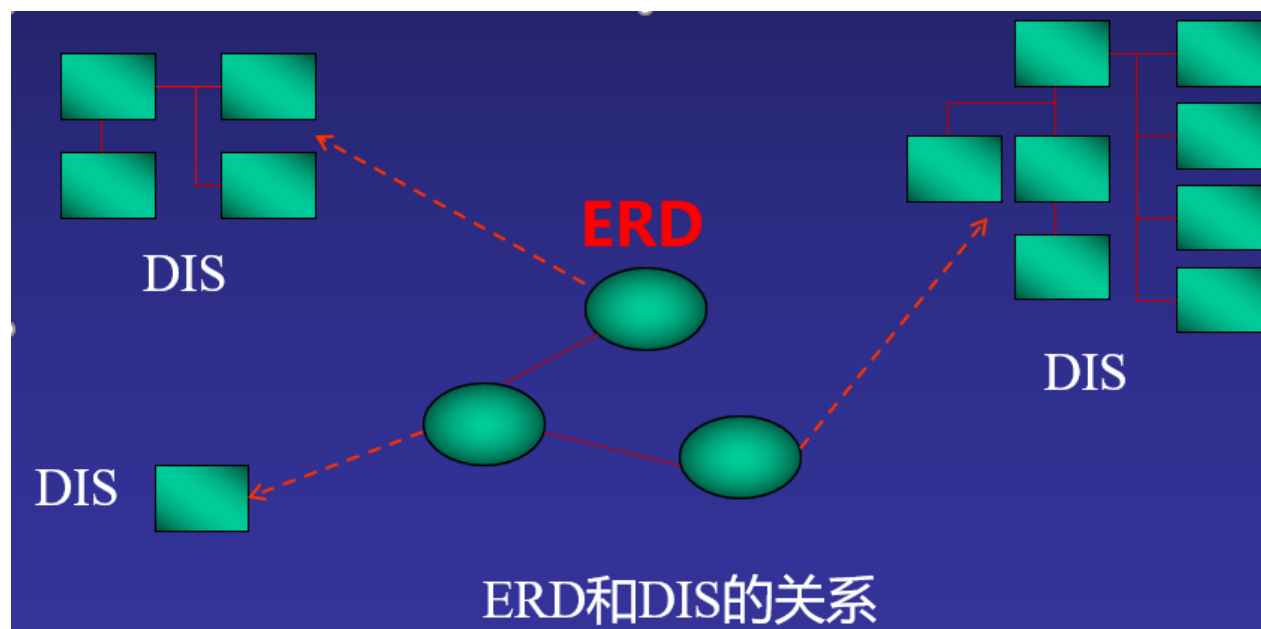
- 物理数据模型

~cont.高级模型



~cont.中级模型

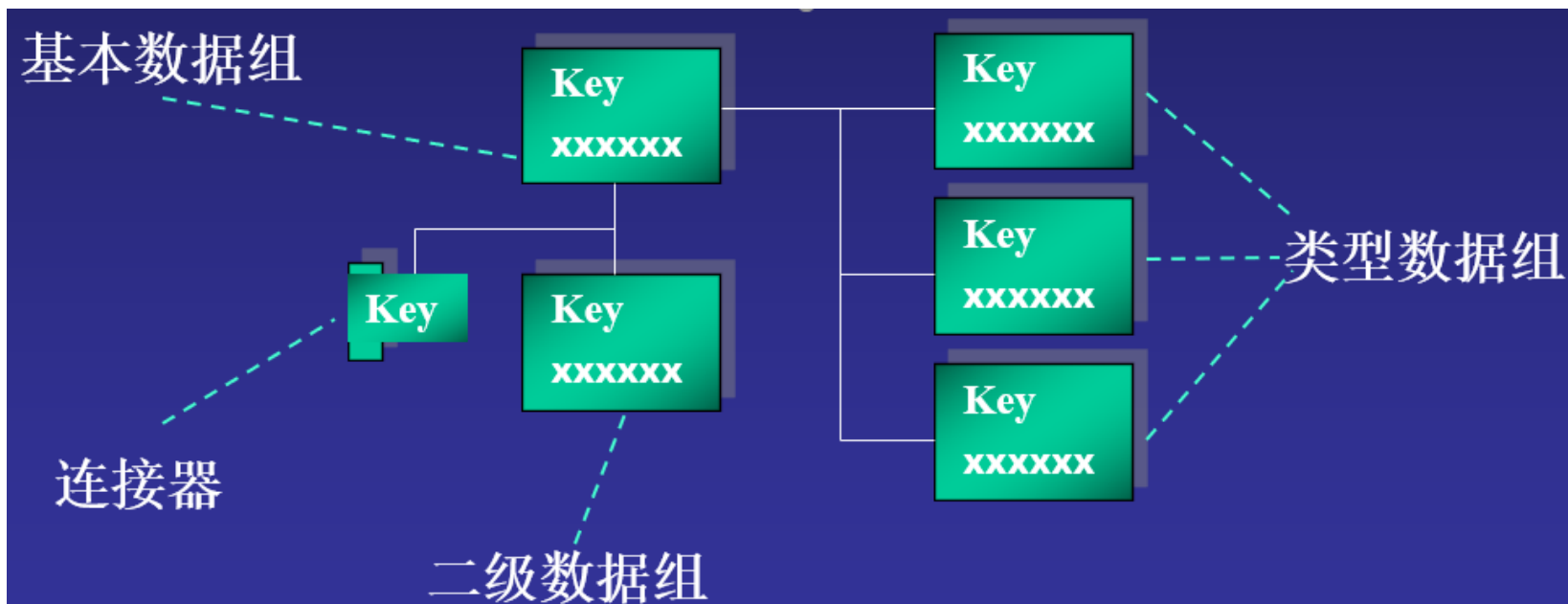
- 中级模型
 - 数据项集（Data item set, DIS）模型
 - E-R图的细分, 每个主题都与一个DIS对应



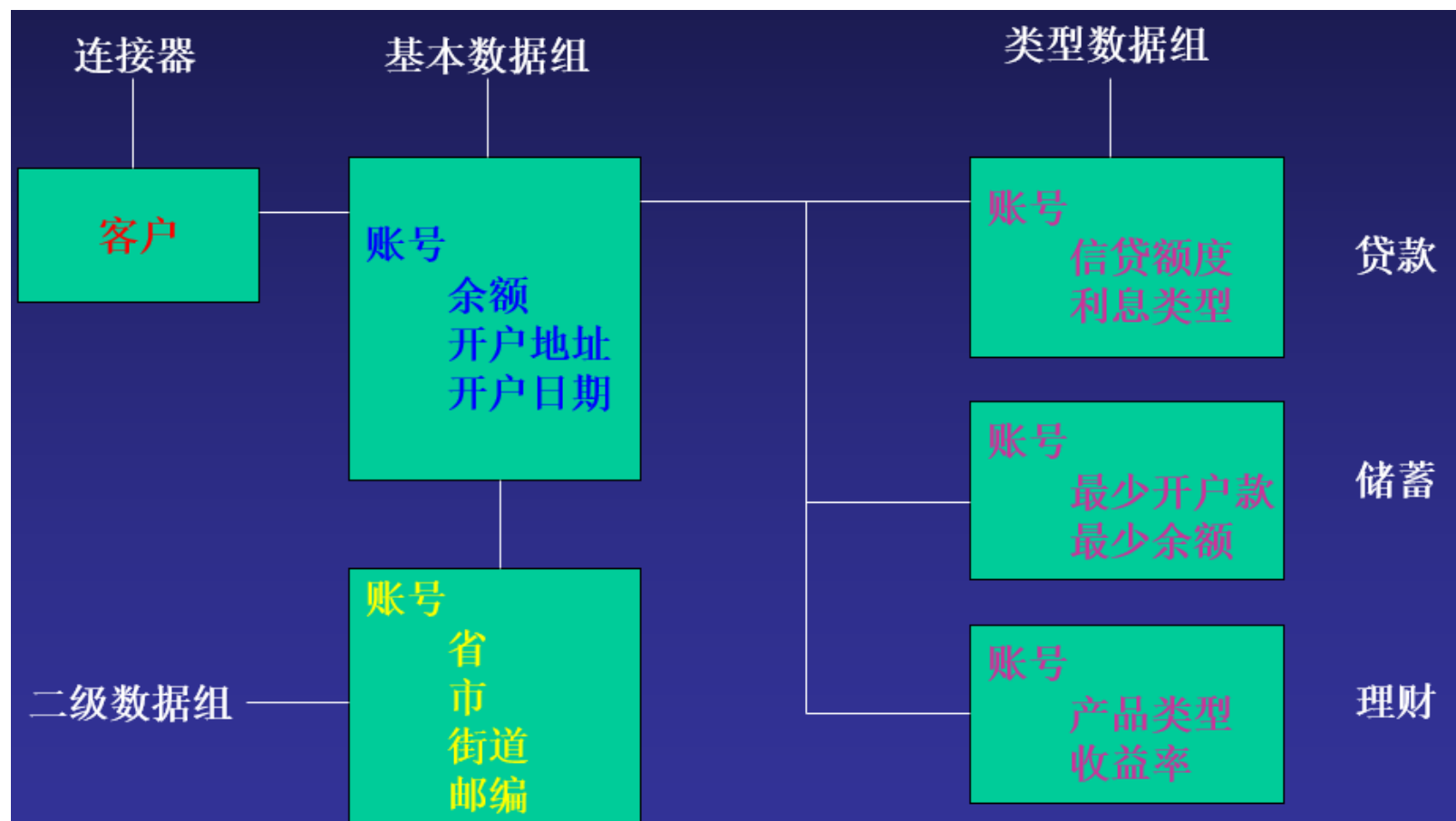
~cont.中级模型

- 中级模型

- DIS中的数据分为4个组别：基本数据组、二级数据组、连接器和类型数据组



~cont. Inmon的三级模型



3.数据仓库的概念模型

- 本节主要内容：
 - 概念模型的定义和主要工作
 - 信息包图
 - 多维模型
 - 维度定义
 - 维度层次
 - 度量和度量值
 - 多维数据集

3.1 概念模型的定义和主要工作

■ 概念模型

- 是数据仓库数据模型的第一层或最高层
- 数据仓库概念模型不如数据库成熟
 - 数据库常见的概念模型：ERD，UML等
 - 数据仓库的大多数业务数据是多维的，但用ERD表示三维以上的数据存在一定困难
- 数据仓库中可以采用信息包图来表示概念模型
 - 信息包图允许用户设计多维信息包并与开发者和其他用户建立联系
 - 这种模型集中在用户对信息包的需要，信息包提供了分析人员思维模式的可视化表示

~cont.

1.界定系统的边界

- 要做的决策类型有哪些?
- 决策者感兴趣的是什么问题?
- 这些问题需要什么样的信息?
- 要得到这些信息需要包含原有DBS哪些部分的数据?

2.确定主要的主题域

- 先确定系统所包含的主题域，再对每个主题域进行明确的描述，内容包括：
 - 主题域的公共码键
 - 主题域之间的联系
 - 充分代表主题域的属性组

~cont.

【例】以商场的DW设计为例，商场经营者最迫切的需求是：准确地把握商场经营情况，跟踪市场趋势，更加合理地制定商品采购与营销政策

1.界定系统的边界

- 需要进行的分析
 - 分析顾客购买趋势; 分析商品供应市场的变化趋势; 分析供应商信用等级
- 需要的数据包括：
 - 商品销售数据; 商品采购数据; 商品库存数据; 顾客信息; 供应商信息
- 通过需求分析，可界定系统边界为：包含原有的销售子系统，采购子系统，库存子系统在内的集合

~cont.

2. 确定主要的主题域

- 主题：商品，供应商，顾客

- 主题之间的联系：

- 商品与供应商

- 每个供应商供应多种商品，一种商品可由多个供应商提供，它们之间的联系是商品供应关系

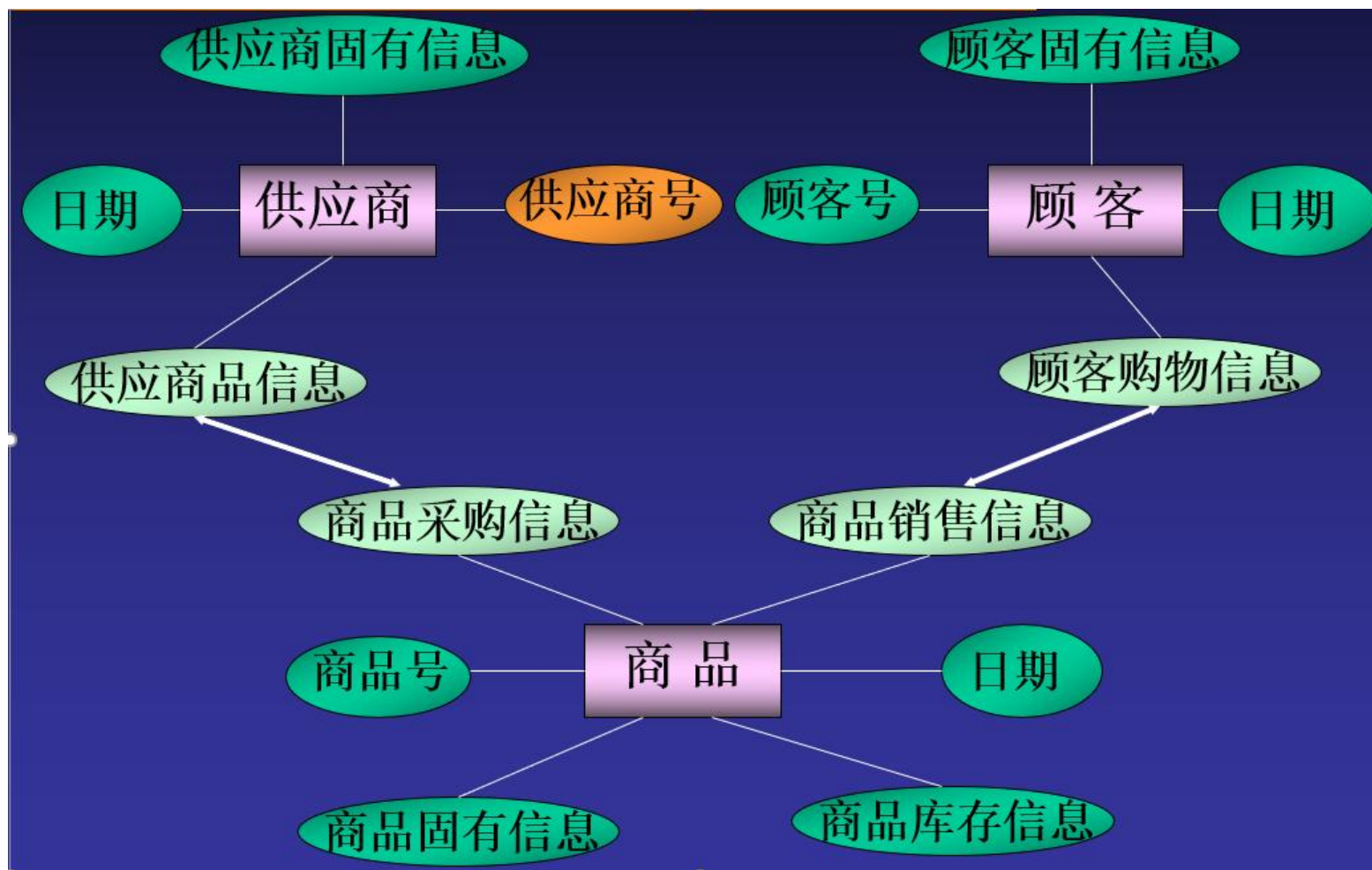
- 商品与顾客

- 一位顾客购买多种商品，一种商品又有多位顾客购买，它们之间的联系是商品销售关系

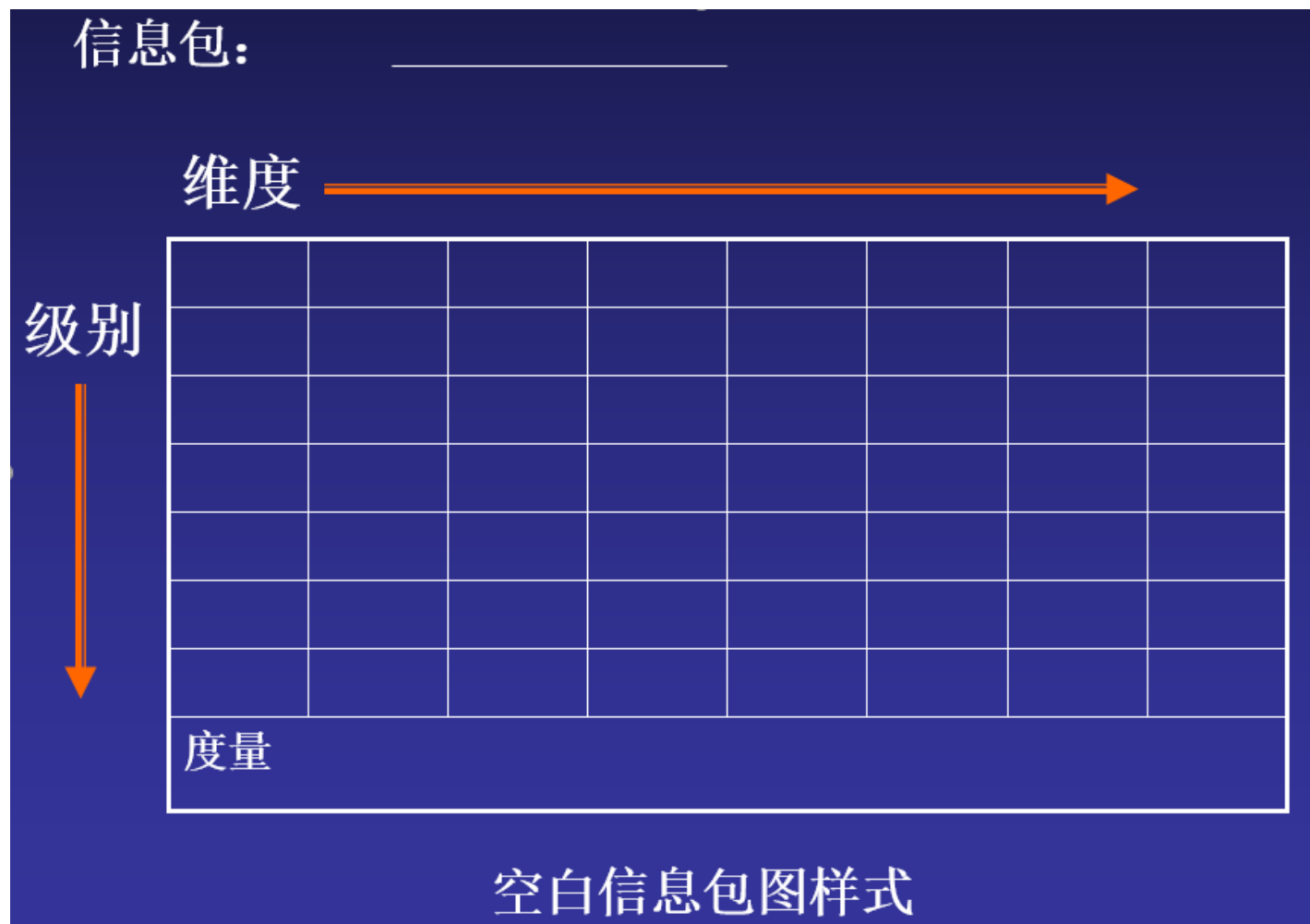
- 顾客与供应商

- 一个顾客可购买不同供应商提供的商品，一个供应商提供的商品可被不同顾客购买，可看出它们间的联系是间接的，通过商品实现

~cont.



3.2 信息包图



~cont.

【例】画出销售分析的信息包图

- 首先根据销售分析的实际需求，确定信息包的**维度**、**类别**和**指标与事实**：
 - **维度**：包括日期维、销售地点维、销售产品维、年龄组别维、性别维等。
 - **级别**：确定各维的详细类别
 - 如：日期维包括年(10)、季度(40)、月(120)等类别
 - 销售地点维包括国家(15)、区域(45)、城市(280)、区(880)、商店(2000)等类别
 - 类似地，可以确定销售产品、年龄组别维、性别维等的详细类别
 - **度量**：确定用于进行分析的数值化信息，包括**预测销售量**、**实际销售量**和**预测偏差**等

~cont.

信息包: 销售分析							
维度							
级别	日期	销售地点	销售产品	年龄组别	性别		
	年(10)	国家(15)	产品类(6)	年龄组(8)	性别组(2)		
	季度(40)	区域(45)	产品组(48)				
	月(120)	城市(280)	产品(240)				
		区(880)					
		商店(2000)					
	度量: 预测销售量、实际销售量、预测偏差						

3.3 多维模型

- 除了采用信息包图表示外，数据仓库的概念模型还可以采用其他模型，如**多维模型**

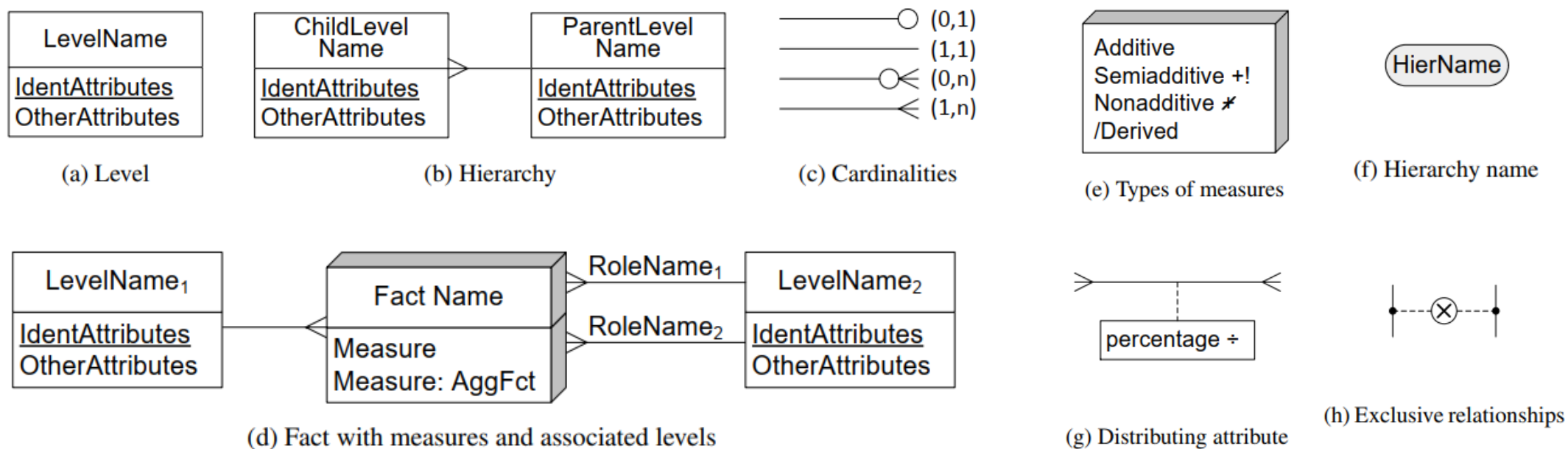


Figure 4.1: Notation of the MultiDim model

~cont.

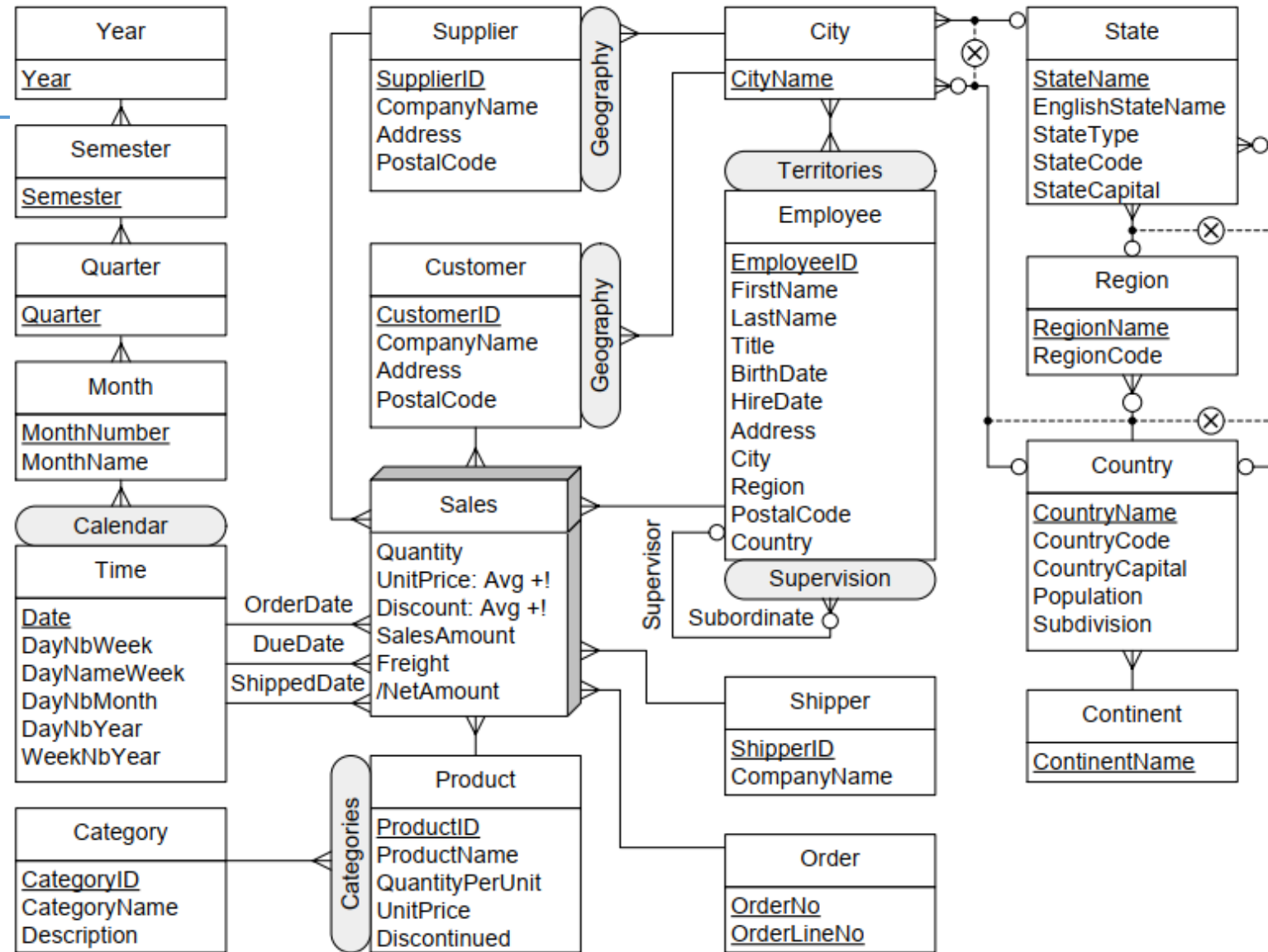
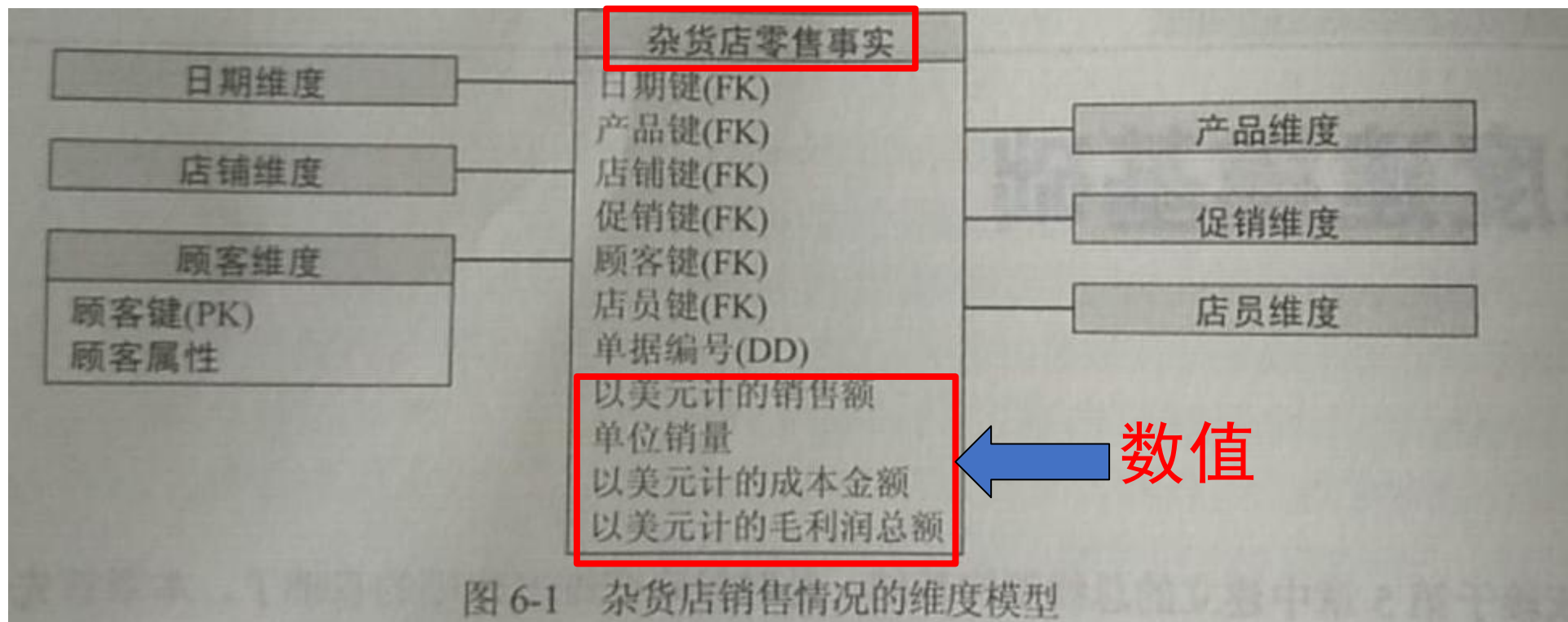


Figure 4.2: Conceptual schema of the Northwind data warehouse

3.4 维度定义

- 维度建模是一个设计准则，具有用于处理现实情况的广泛技术组合
- 维度建模首先从将现实情况划分为度量和上下文(context)开始
 - 度量通常是数值的且会重复使用
 - 数值测量值就是事实(fact)
 - 事实是非常具体的、精心定义的数值属性，而围绕事实的上下文则是开放且详尽的

~cont.



~cont.

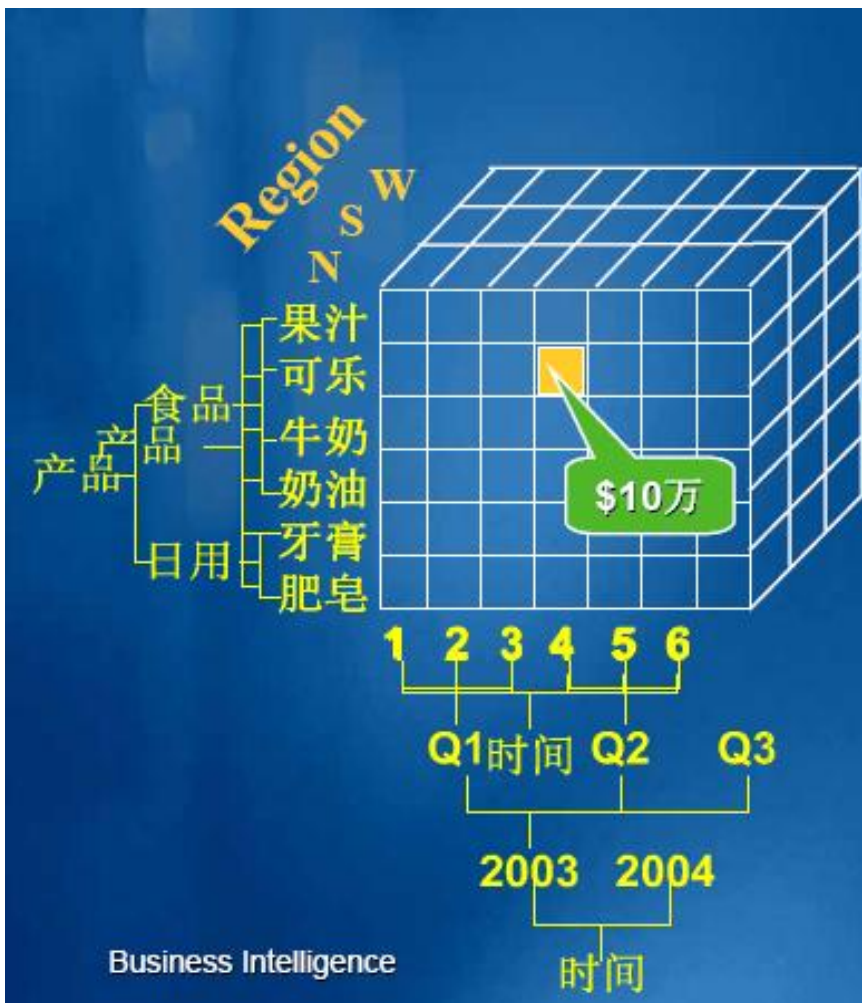
■ 维度的定义

- 维度(dimension)是一个物理特性（如时间、地点、产品等），它是表达数据仓库中信息的一个基本途径，用作数据分析的视角，如，从时间维分析销售的情况
 - 通常的报表只包含有行和列两维，但在数据仓库中所存储的数据大多是用多维(三维或三维以上)视图表示的
- 维度也是多维数据集的结构特性。它们是事实数据表中用来描述数据的分类和有组织的层次结构（级别）
 - 这些分类和级别描述了一些相似的成员集合

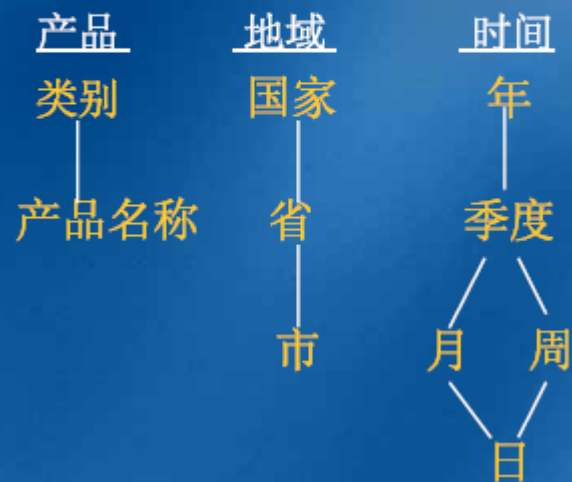
■ 维度具有原子性，即不可分

~cont.

■ 例1:



维度的层次概念:



03年4月份在北方卖掉了价值
十万元的可乐

~cont.

■ 例2:

关系数据集的记录视图:

产品	时间	地区	销量
VCD机	2002.3.10	北京	5728
传真机	2002.3.10	北京	6739
刻录机	2002.3.10	北京	3976
复印机	2002.3.10	北京	6395
打印机	2002.3.10	上海	6365
传真机	2002.3.10	上海	6744

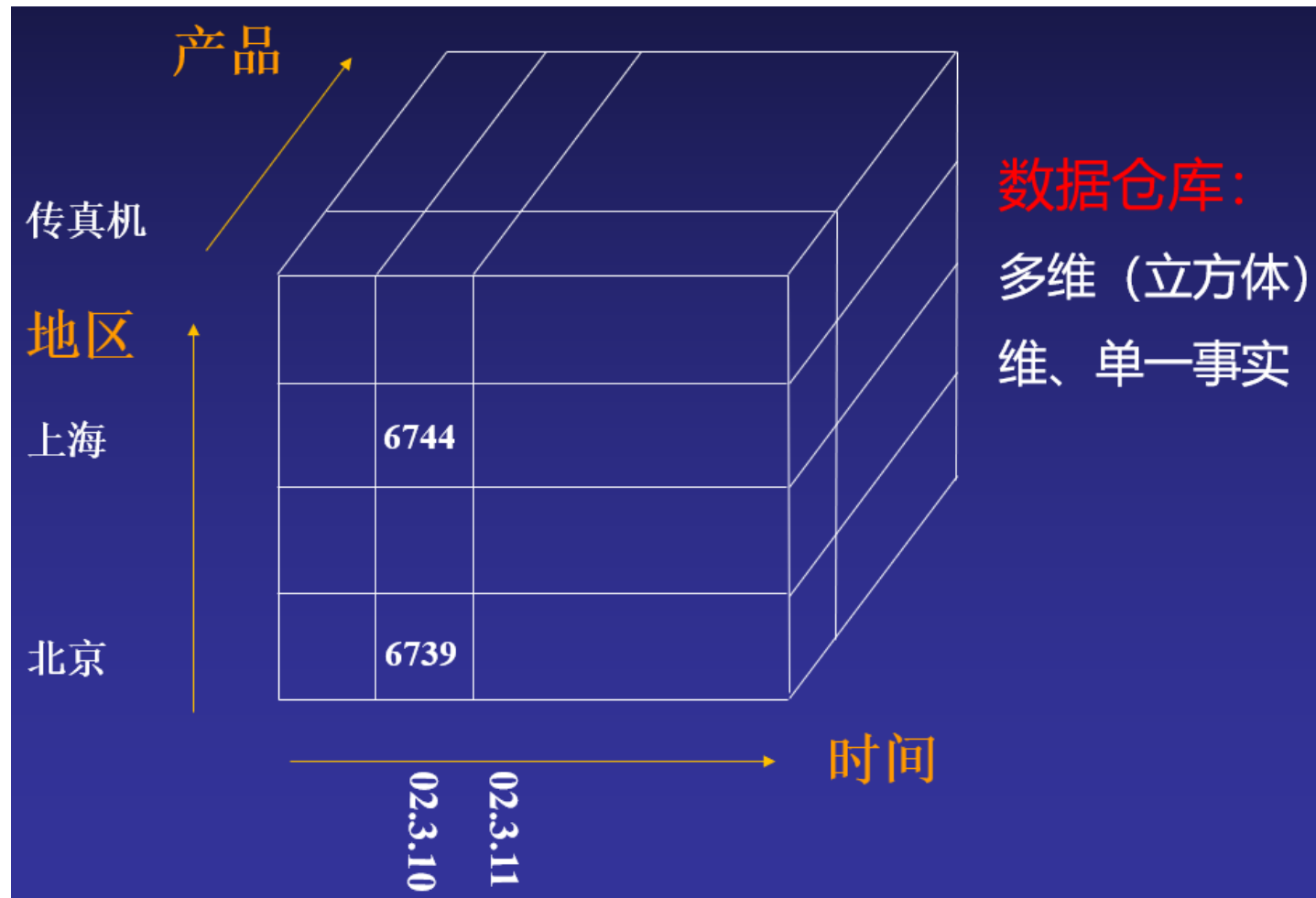
传统型:

二维 (关系型)

记录、字段

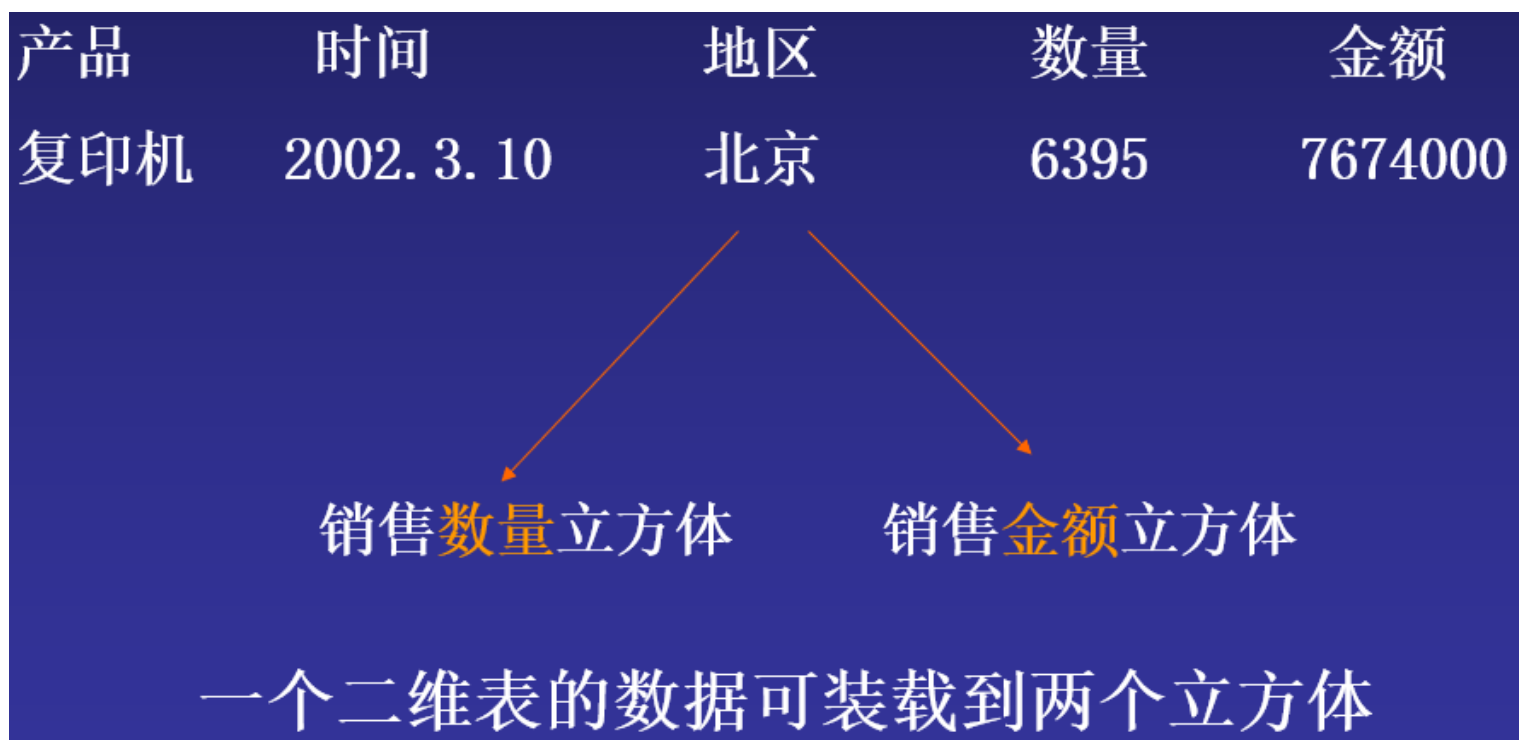
~cont.

■ 多维数据集视图：



~cont.

- 传统型的一个二维表，在数据仓库中可能需要多个立方体来表示
- 数据仓库的一个立方体可能由多个传统型的表组成



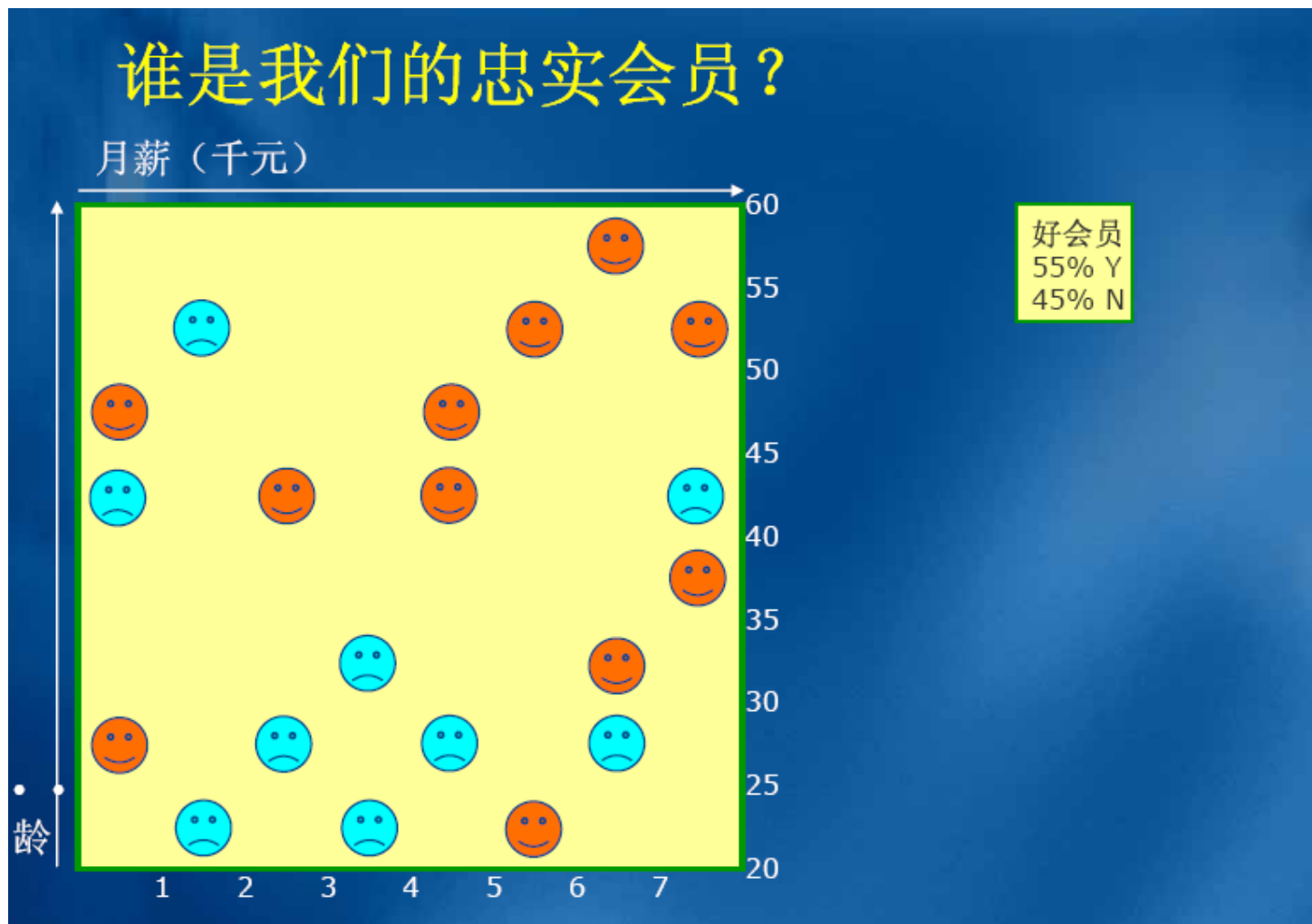
维度的分解与合成

- 维度的分解与合成是在一个维度内进一步细分数据或将数据按照另一标准组合的过程
 - 例如，当以地理位置维观察数据时，用户可以首先以国家（如中国）为单位观察数据，然后可以选择观察某一个地区（如华东地区）的数据，接下来可以选择观察某一个省或城市(如上海)的数据，这就是数据分解的过程
- 维度的合成是维度分解的逆过程
 - 例如用户开始以省市为观察对象，接着再以地区、国家等为观察对象，就是一个数据合成的过程

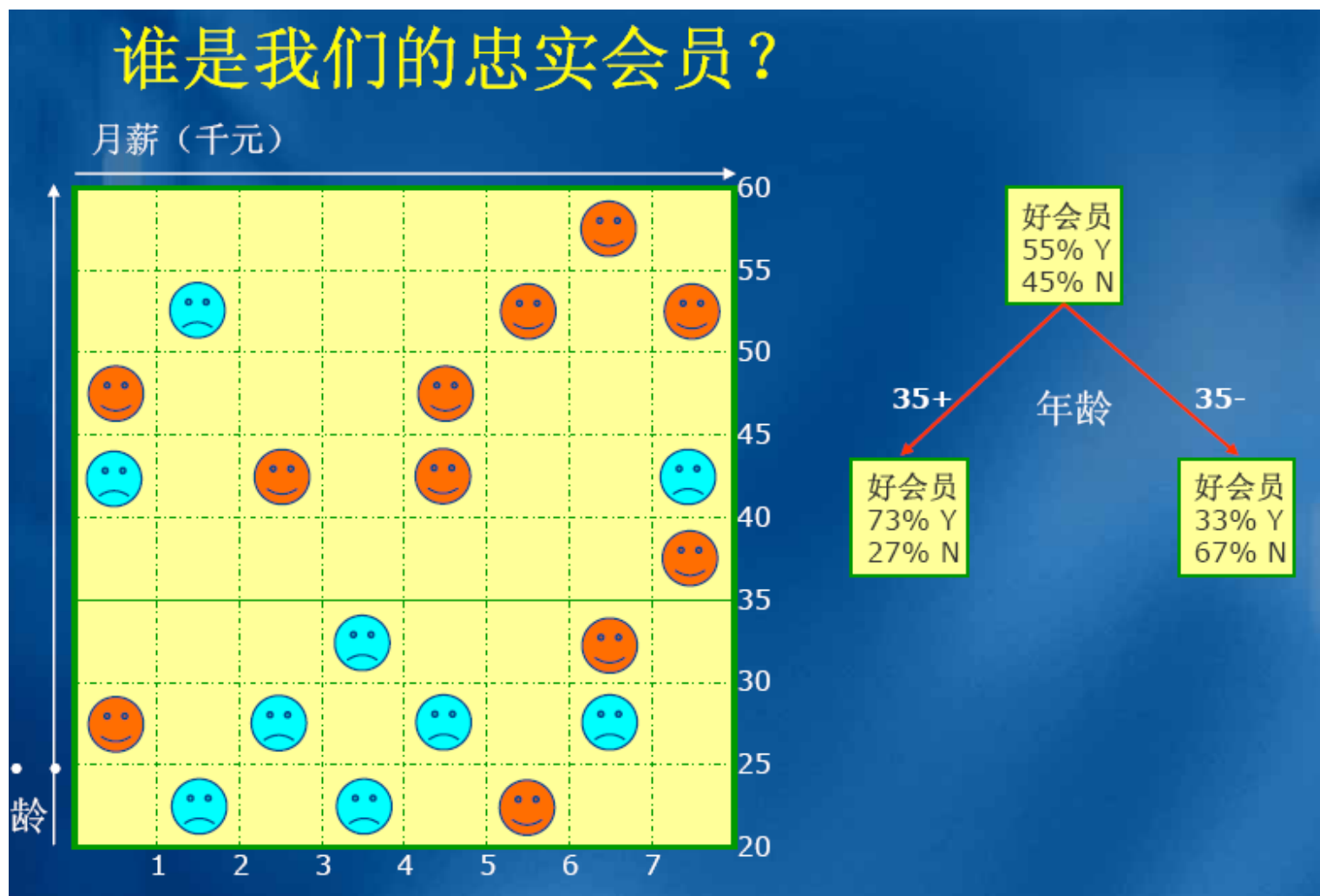
维度分解的例子

- 我们有大量的会员
 - 年龄在20 – 60 岁
 - 月薪在0 – 8000 元
 - 55% 的会员被我们认可为忠实会员（好会员）
 - 55%会员都是些什么会员？

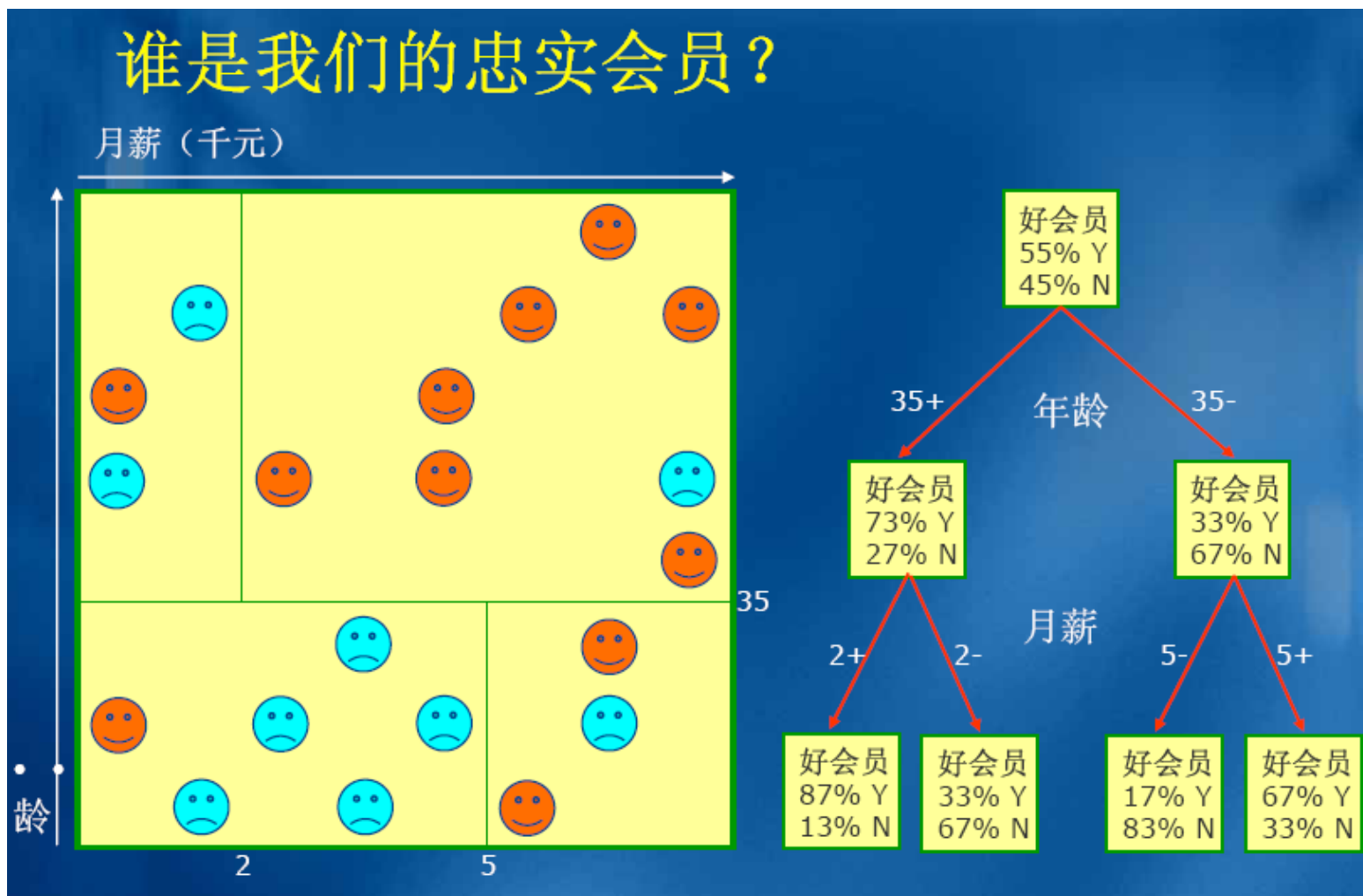
~cont.



~cont.



~cont.



维度合成的例子

- 加拿大某大学数据库

Name	Status	Major	BirthPlace	GPA
Anderson	M.A.	history	Vancouver	3.5
Bach	Junior	math	Calgary	3.7
Carlton	Junior	liberal art	Edmonton	2.6
Fraser	M.S.	physics	Ottawa	3.9
Gupta	Ph.D.	math	Bombay	3.3
Hart	Sophomore	chemistry	Richmond	2.7
Jackson	Senior	computing	Victoria	3.5
Liu	Ph.D.	biology	Shanghai	3.4
...	...	...	...	...
Meyer	Sophomore	music	Burnaby	3.0
Monk	Ph.D.	computing	Victoria	3.8
Wang	M.S.	statistics	Nanjing	3.2
Wise	Freshman	literature	Toronto	3.9

~cont.

{Bumaby, ..., Vancouver, Victoria} $\subset$ British Columbia

{Calgary, ..., Edmonton, Lethbridge} $\subset$ Alberta

{Hamilton, Toronto, Waterloo} $\subset$ Ontario

{Bombay, ..., New Delhi} $\subset$ India

{Beijing, Nanjing, ..., Shanghai} $\subset$ China

{India, China} $\subset$ foreign

{ British Columbia, Alberta, ..., Ontario} $\subset$ Canada

{foreign, Canada} $\subset$ ANY(place)

根据BirthPlace的合成

~cont.



~cont.

{biology,chemistry,computing,...,physics} $\subset$ science

{literature,music,...,painting} $\subset$ art

{science, art} $\subset$ ANY(major)

} 根据Major进行的合成

{freshman,sophomore,junior,senior} $\subset$ undergraduate

{M.S.,M.A.,Ph.D.} $\subset$ graduate

{undergraduate,graduate} $\subset$ ANY(status)

} 根据Status进行的合成

{0.0-1.99} $\subset$ poor

{2.0-2.99} $\subset$ average

{3.0-3.99} $\subset$ good

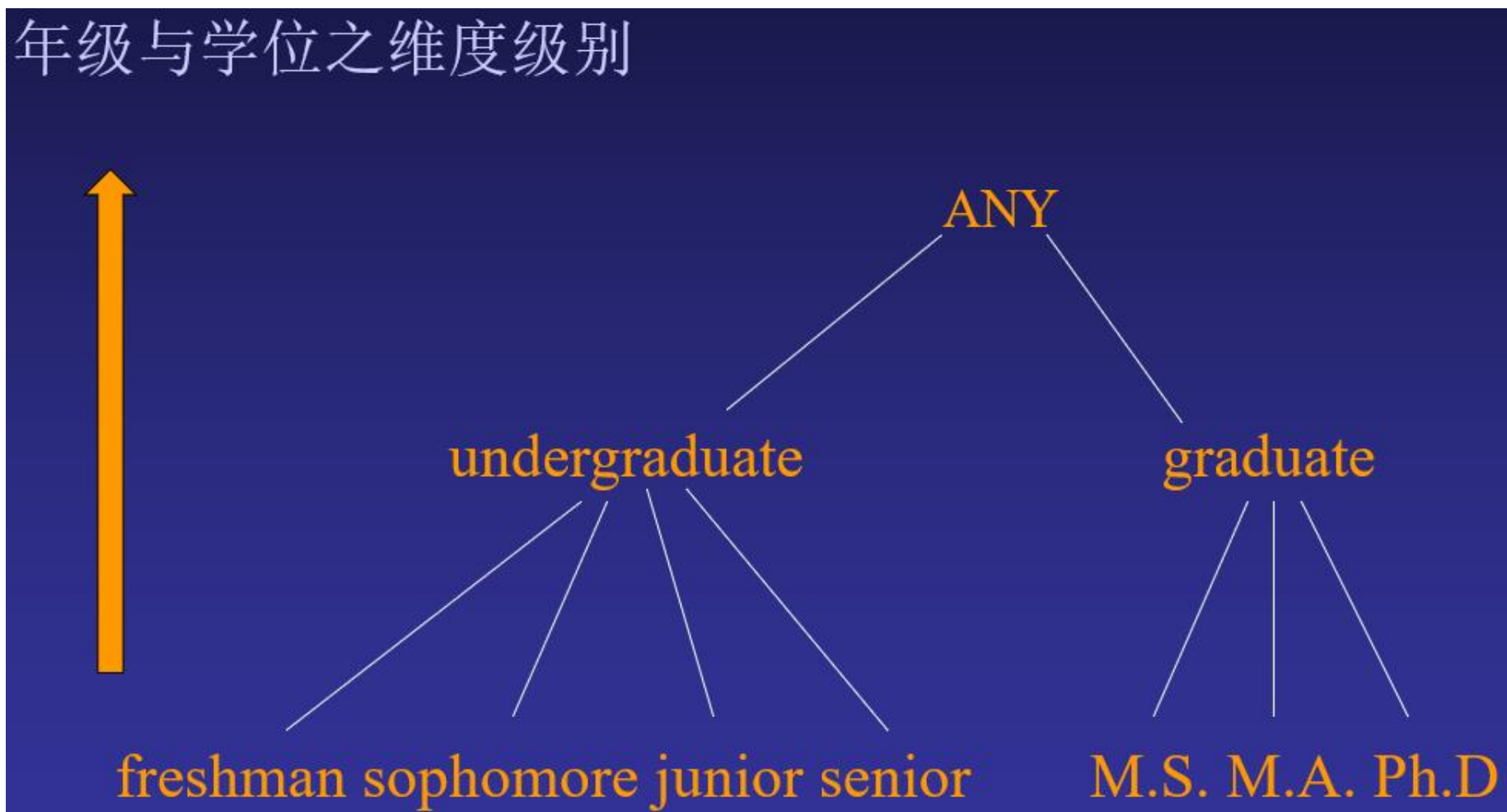
{4.0-4.99} $\subset$ excellent

{poor,average,good,excellent} $\subset$ ANY(grade)

} 根据GPA进行的合成

~cont.

年级与学位之维度级别



~cont.

- 将研究生数据合成出来

Names	Major	BirthPlace	GPA	Vote
Anderson	history	Vancouver	3.5	1
Fraser	physics	Ottawa	3.9	1
Gupta	math	Bombay	3.3	1
Liu	biology	Shanghai	3.4	1
...	...	...	...	...
Monk	computing	Victoria	3.8	1
Wang	staistics	Nanjing	3.2	1

维度合成的策略

■ 策略1：属性移除(Attribute Removal)

- 如，**Names**这个属性中有许多不同的属性值，且没有较高的概念层级可以表示它，所以Names属性可以被移除

Major	BirthPlace	GPA	Vote
history	Vancouver	3.5	1
physics	Ottawa	3.9	1
math	Bombay	3.3	1
biology	Shanghai	3.4	1
...	...	...	...
computing	Victoria	3.8	1
statistics	Nanjing	3.2	1

~cont.

■ 策略2：维级别的爬升

– 某一属性在概念阶层中存在着一个更高层级的概念，则该属性值就以其更高层级的值來取代，如：

- “history” 、 “physics” 、 “math” 、 “biology” 会由 “science” 取代
- “literature” 、 “ music” 、 “painting” 会由 “art” 取代

~cont.

■ 策略3：数据计数的传播(vote propagation)

- 属性值向上爬升后，若产生相同的 tuple，则将相同的 tuple 合并为一条一般化tuple，并将 vote 值累加到归纳后的tuple 中

Major	Birth_Place	GPA	Vote
art	B.C	excellent	35
science	Ontario	excellent	10
science	B.C	excellent	30
science	India	good	10
science	China	good	15

~cont.

Major	Birth_Place	GPA	Vote
Art	Canada	excellent	35
Science	Canada	excellent	40
Science	Foreign	good	25



Major	Birth_Place	GPA	Vote
{art,science}	Canada	Excellent	75
Science	Foreign	good	25

3.5 维度层次

- 维层次结构(Dimension hierarchy)是维度中成员_{成员}的集合以及这些成员之间的相对位置
- 一个维层次结构中的5个基本概念：
 - Dimension：包括描述它的属性_{属性}
 - Member成员：维的实例_{维的实例}
 - Level层级：代表粒度_{代表粒度}
 - Granularity粒度：细节层次_{细节层次}
 - 维度模型(dimension schema)：一个维的整个层次结构(各个Level及其成员)

• 数据仓库中dimension,level,hierarchy的理解：<https://blog.csdn.net/Athenaer/article/details/7924278>

~cont.

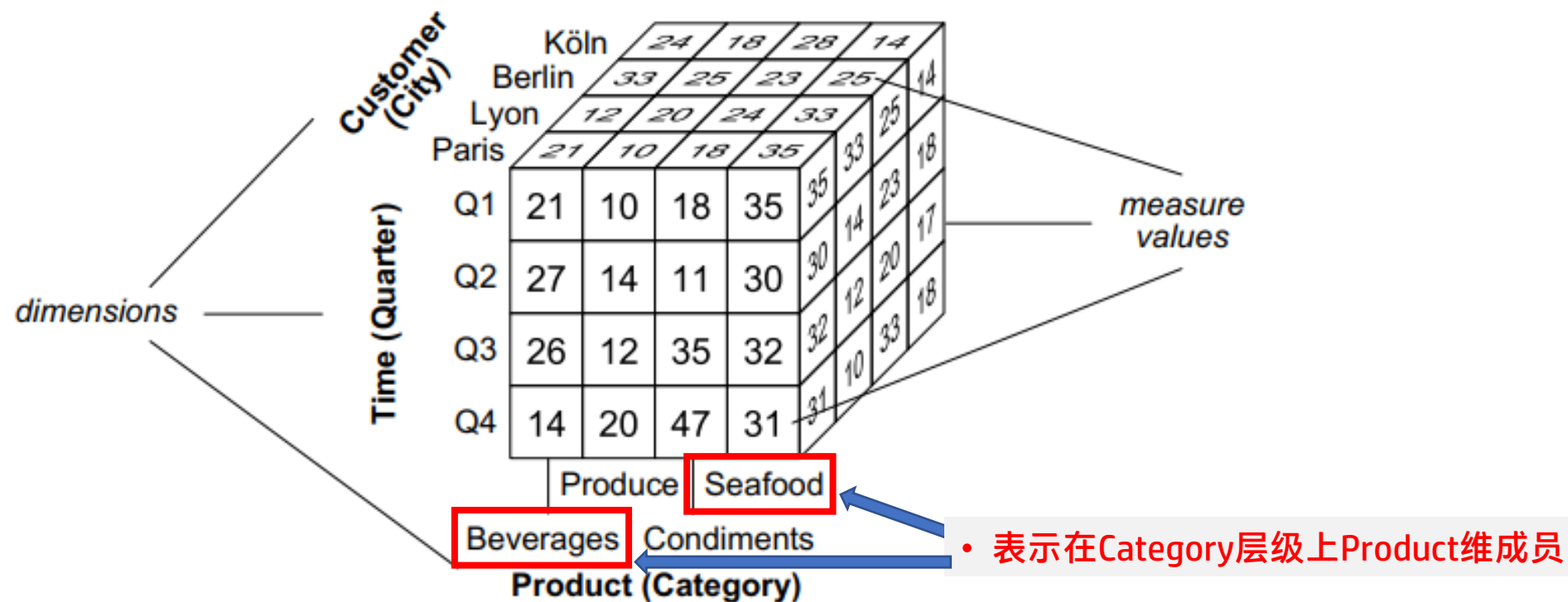


Fig. 3.1 A three-dimensional cube for sales data with dimensions Product, Time, and Customer, and a measure Quantity

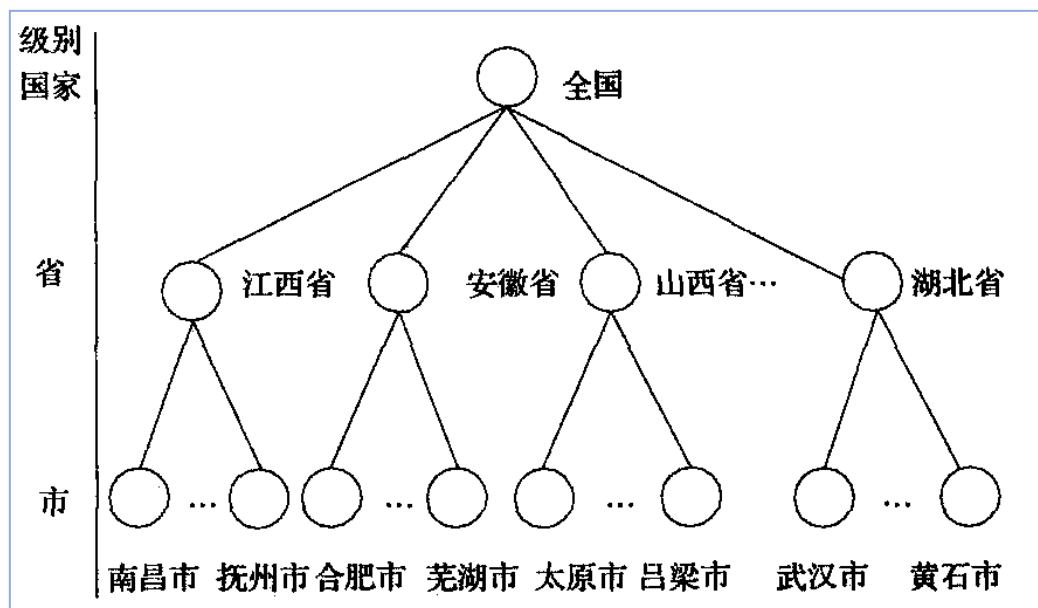
- Product维可能包括属性: ProductNumber和 UnitPrice

~cont.

■ 维度层次结构的类型

— 正常的层次结构

- 正常层次结构指的是**齐整**和**平衡**的层次结构：它的所有层次分支都将降至具有相同级别的最低层次, 并且每个成员的**逻辑父代**(Logical Parent)直接位于该成员的上一级别

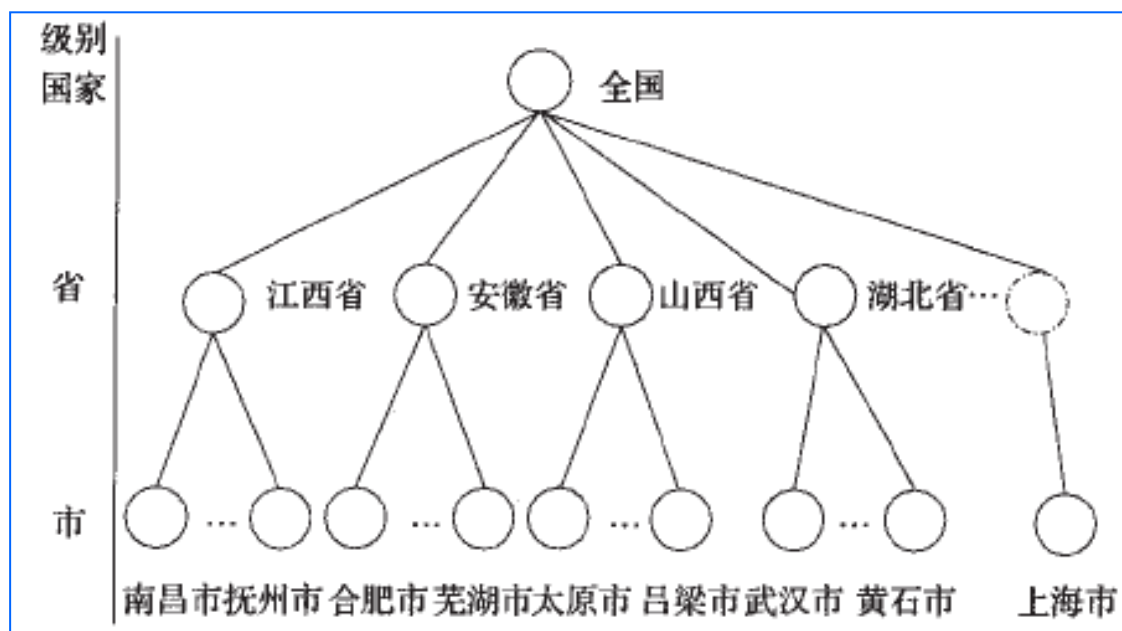


~cont.

■ 维度层次的结构类型

— 不齐整层次结构

- 在不齐整层次结构中, 各层次分支从高层遍历到最低层所跨过的层次级别会有所不同, 也就是说, 至少有一个成员的逻辑父级不同于其它成员的父级

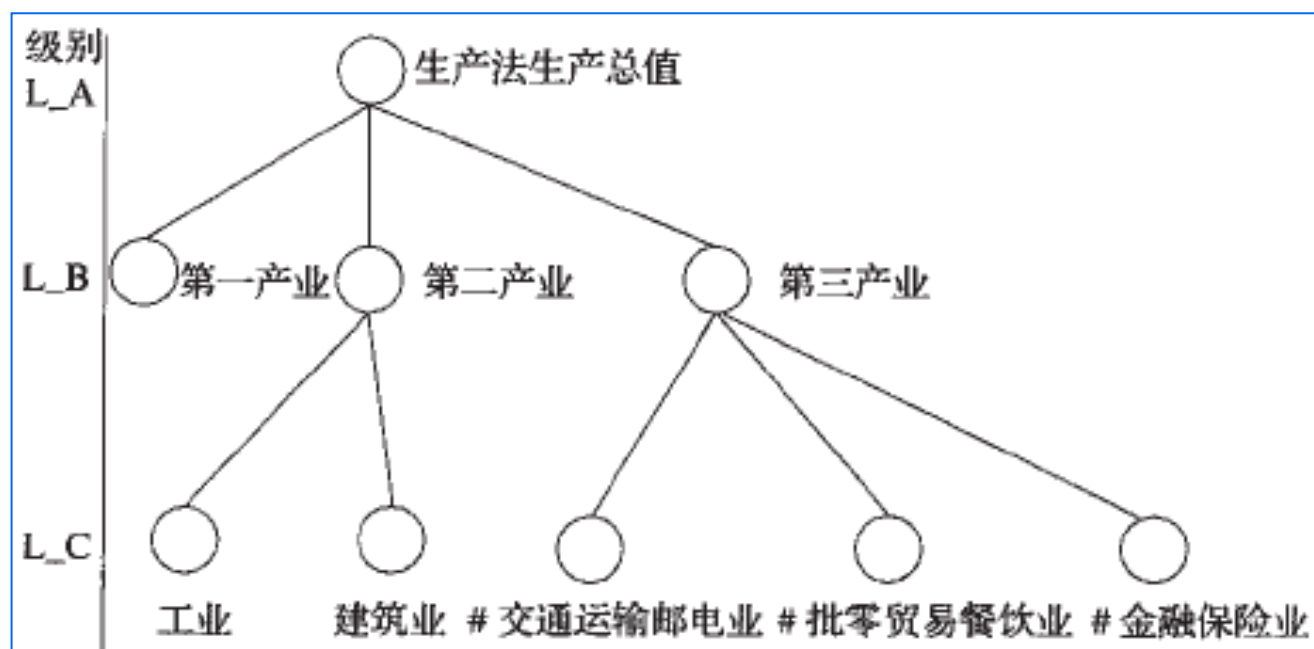


~cont.

■ 维度层次的结构类型

— 不平衡层次结构

- 不平衡层次结构指的是各层次结构分支从高层往低层的遍历过程中将降至具有不同级别的最低层次



~cont.

- 维度层次的结构类型

- 具有多个层次结构的维度

- 维度内的成员可以按照两个或者两个以上的层次结构进行抽象综合，这些不同的层次结构都是基于成员内在属性的
 - 例如商品维，即可以按照商品的种类，也可以按照商品的产地进行层次化

3.6缓慢变化的维

- 现实世界中很多情况下维并不总是稳定不变的，而是缓慢变化的。这种变化可能发生在：

- 结构层次at the structure level

- 如，删除数据源中属性A > A不可用 > 相应地删除维表中的属性A  维表结构发生变化

- 实例层次at the instance level

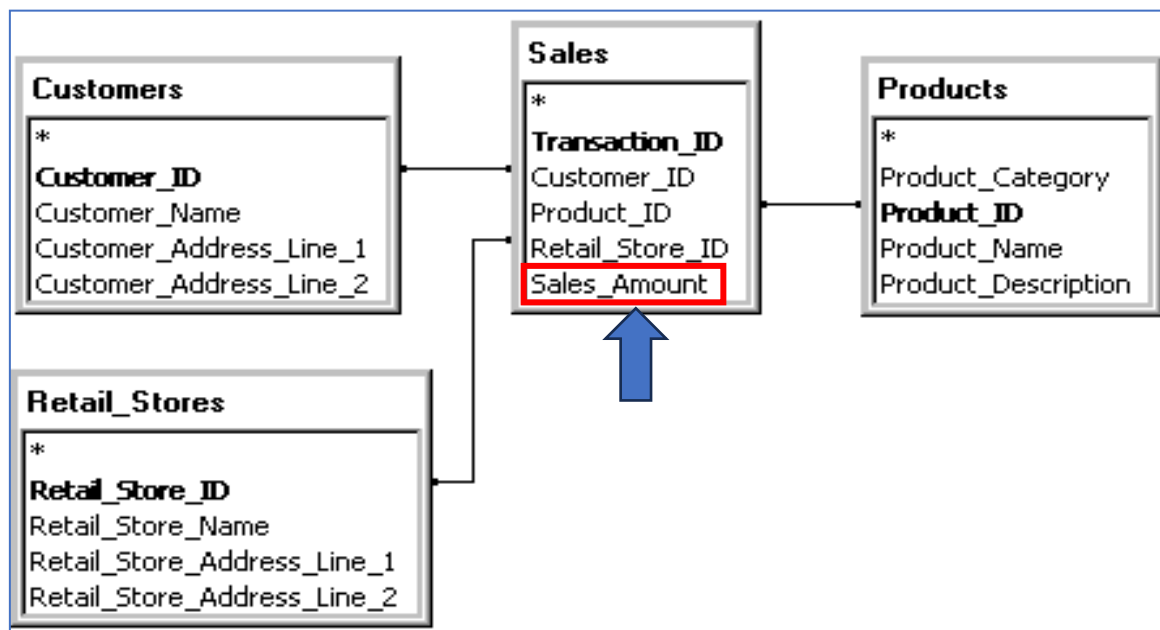
- 如，由于错误，维表数据需要更正，使用新值替代旧值；或分析场景的上下文条件发生变化，维表内容必须做相应改变


维表内容发生变化

- 参考资料：https://blog.csdn.net/weixin_44318830/article/details/105961276

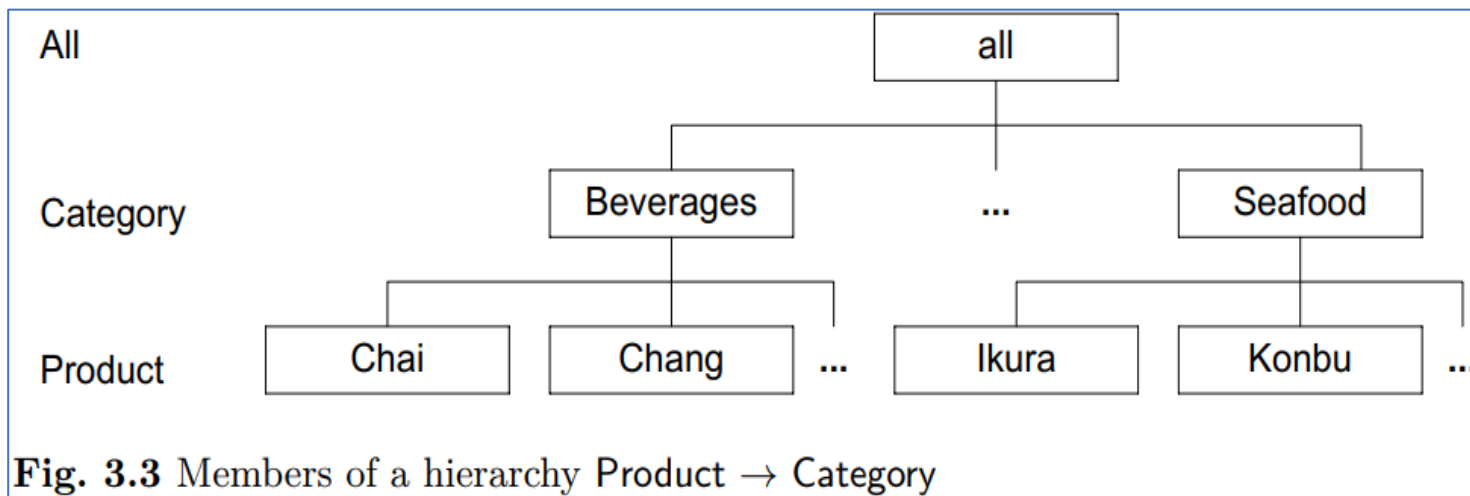
3.7 度量和度量值

- 度量值(measure value)是一组值，基于事实数据表中的一列，而且通常为数值（但并不总是这种情况）
 - 如一个多维数据集具有以下架构和单个度量值 Sales(基于 Sales 事实数据表中的 Sales_Amount 列)



汇总

- **汇总**(Summarizability)是度量的一种主要运算。要确保汇总运算正确，需要一些条件，其中包括：
 - **实例的不相交性**
 - 某层级的实例相对于父级进行分组时，必须形成**互不相交的子集**



- 一个product不能属于两个不同的category，否则汇总时就会被计算两次

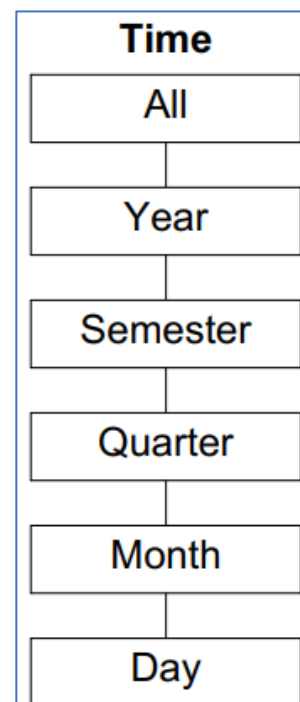
~cont.

- **汇总**(Summarizability)是度量中的一种主要运算。要确保汇总运算正确，需要一些条件，其中包括：

- 实例的不相交性

- 完全性

- 所有实例必须包含在层次结构Hierarchy中，
- 且每个实例必须关联一个父层级



- 在Time的层次结构中，必须包含感兴趣的所有的天Day，每天必须归属于某个月

~cont.

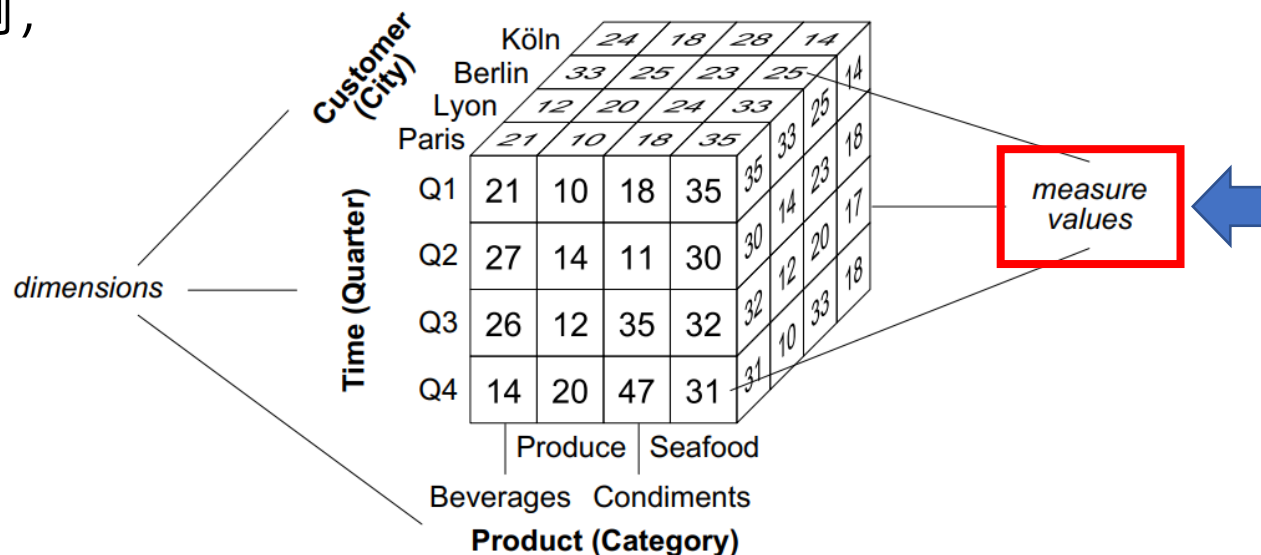
- **汇总**(Summarizability)是度量的一种主要运算。要确保汇总运算正确，需要一些条件，其中包括：
 - 实例的不相交性
 - 完全性
 - 正确性
 - 指聚集函数的正确使用
 - 不同类型的度量使用不同类型的聚集函数

度量类型

■ 根据聚集方式的不同可将度量分为以下三类：

– 加性度量additive measures

- 可沿**所有维**执行加法，是最常见的一种类型
- 例，



- 当遍历Product, Time, 和Customer维的时候可以执行加法，这种measures就是加性度量

Fig. 3.1 A three-dimensional cube for sales data with dimensions Product, Time, and Customer, and a measure Quantity

~cont.

- 根据聚集方式的不同可将度量分为以下三类：
 - 可加性度量additive measures
 - 半加性度量semiadditive measures
 - 可沿一些但非全部维执行有意义的加法
 - 例，库存数量无法在时间维度上有意义地汇总，如，将两个不同季度的库存数量相加
 - 不可加性度量nonadditive measures
 - 在任何维上不能执行有意义的加法
 - 典型例子：物品单价，单位成本，交换率

3.8 多维数据集

- 多维数据集是包含维度和度量值的多维结构（表现为立方体cube）
- 维度定义了多维数据集的结构，而度量值则提供了最终用户感兴趣的数值
- 多维数据集提供了一种便于使用的查询数据的机制，不但快捷，而且响应时间一致
- 单元(cell)是多维数据集的原子元素，代表一个事件event
 - 它是该多维数据集相关联的每个维度的某个成员的唯一逻辑交集

~cont.

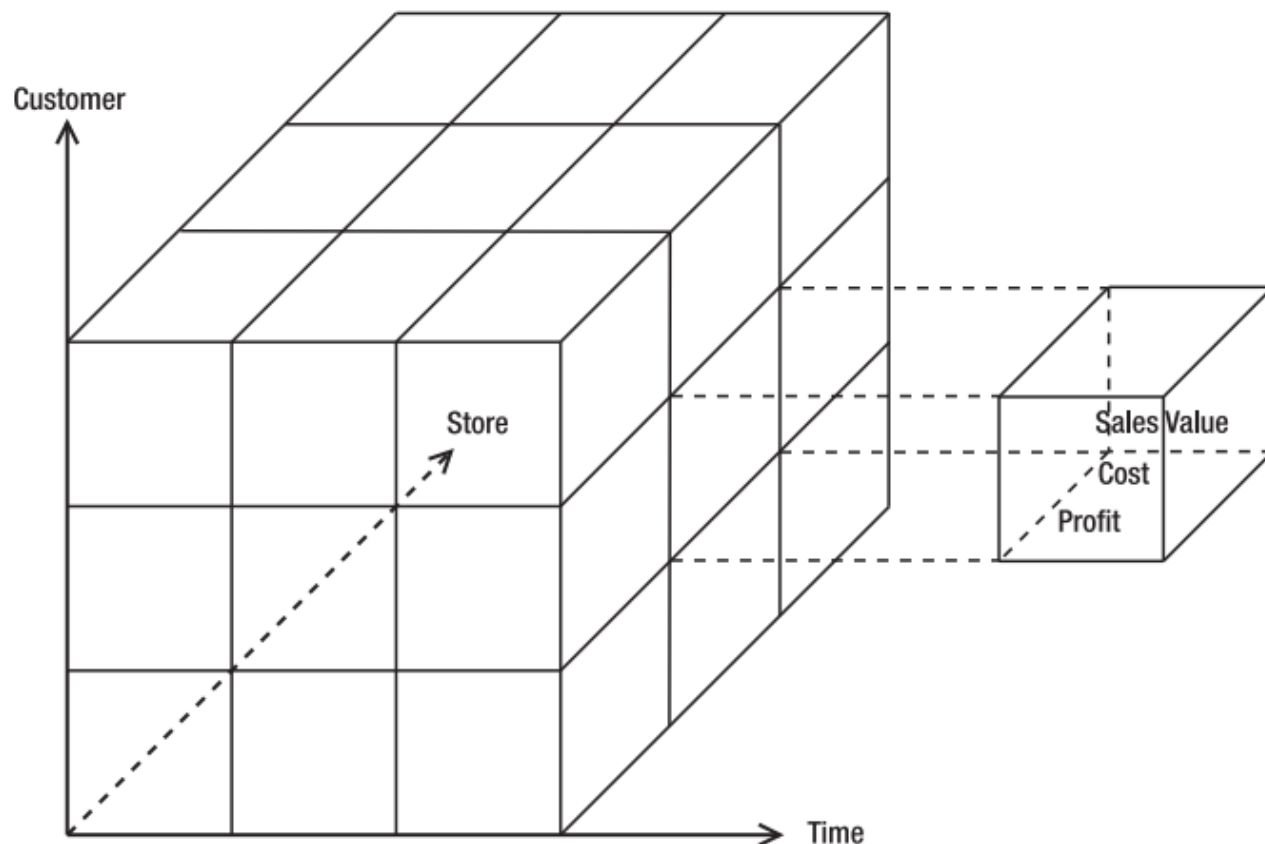
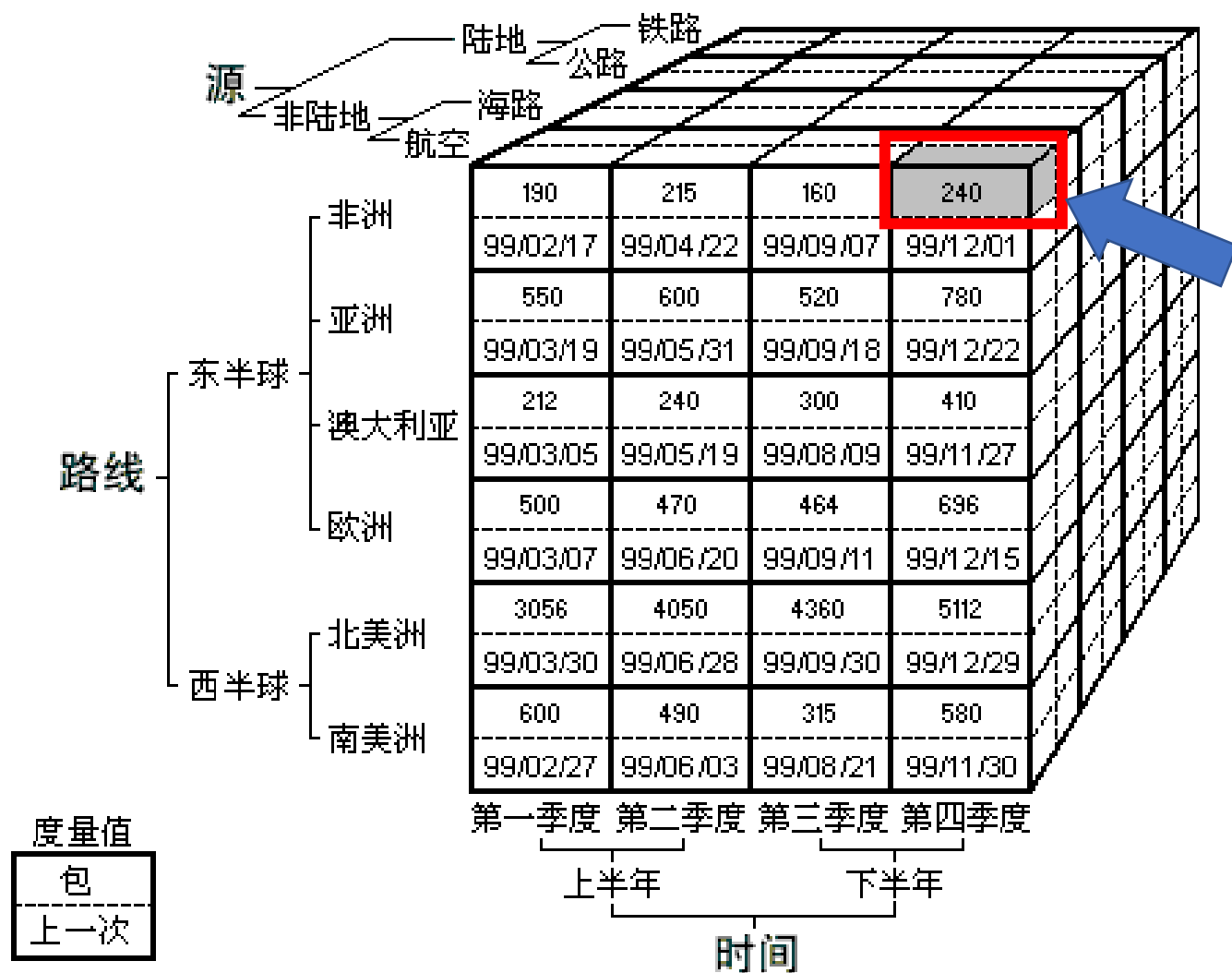


Figure 1-2. A cube with three dimensions

- 假设每个维(Time, Customer, Store)有100个段segments, 那么该立方体包含 $100 \times 100 \times 100 = 1$ 百万个单元
- 一个单元表示事件: 一个顾客于某时间在某店购买东西
- 单元可能包括以下三个数据:
 - 销售值sales value
 - 成本cost
 - 利润Profit

~cont.

多维数据集有一个带阴影的单元



- “源”维度的“航空”成员
- “路线”维度的“非洲”成员
- “时间”维度的“第四季度”成员
- “度量值”中的“包”成员

4.数据仓库的逻辑模型

- 逻辑模型
 - 是数据仓库数据模型的第二层
 - 与传统关系模型相比，它简化了用户分析所需的关系，进而数据结构也得到简化
 - 从支持决策的角度去定义数据实体，更适合大量复杂查询
 - 针对每个主题，有多个维度表，但是只有一个事实表
 - 通常有三种逻辑模型表示法：
 - 星型模型(star schema)，雪花模型(Snowflake schema)和星座模型(galaxy schema)
- 工作
 - 分析主题，定义逻辑模型（星型模型或雪花模型）
 - 确定数据粒度的选择
 - 确定数据分割策略
 - 如需要，增加导出字段等

4.1 星型模型

- 星型模型包含三种逻辑实体：

- 维度表

- 逻辑上通过key关联
 - 维度表往往包含相当多的属性
 - 典型的属性是文本的、离散的
 - 维度往往含有层次（级别）
 - 主键（primary key）往往是系统产生的
 - 很可能是共享的

- 事实表

- 对应联系

~cont.

- 星型模型包含三种逻辑实体：

- 维度表

- 事实表

- 一个数据仓库主题模型只有一个事实表
 - 事实表经常有百万行或更多
 - 事实往往是数字“度量”
 - 有些事实可以被累加，另一些不能
 - 最小粒度原则
 - 不欢迎描述性属性（瘦高vs. 矮胖）

- 对应联系

~cont.

- 星型模型包含三种逻辑实体：

- 维度表

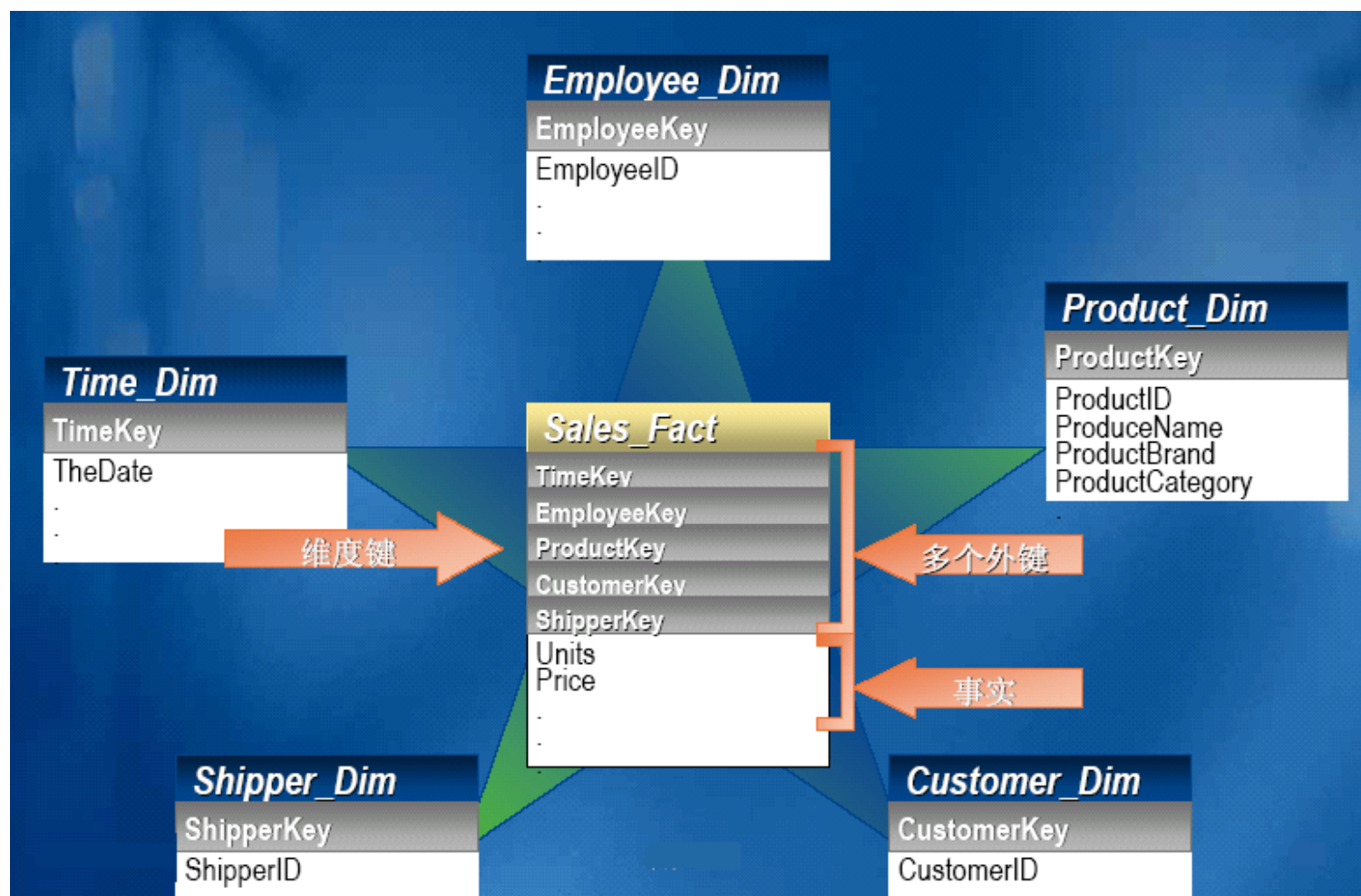
- 事实表

- 对应联系

- 一个事实表对应多个不同维表
 - 一个维表可对应不同的事实表

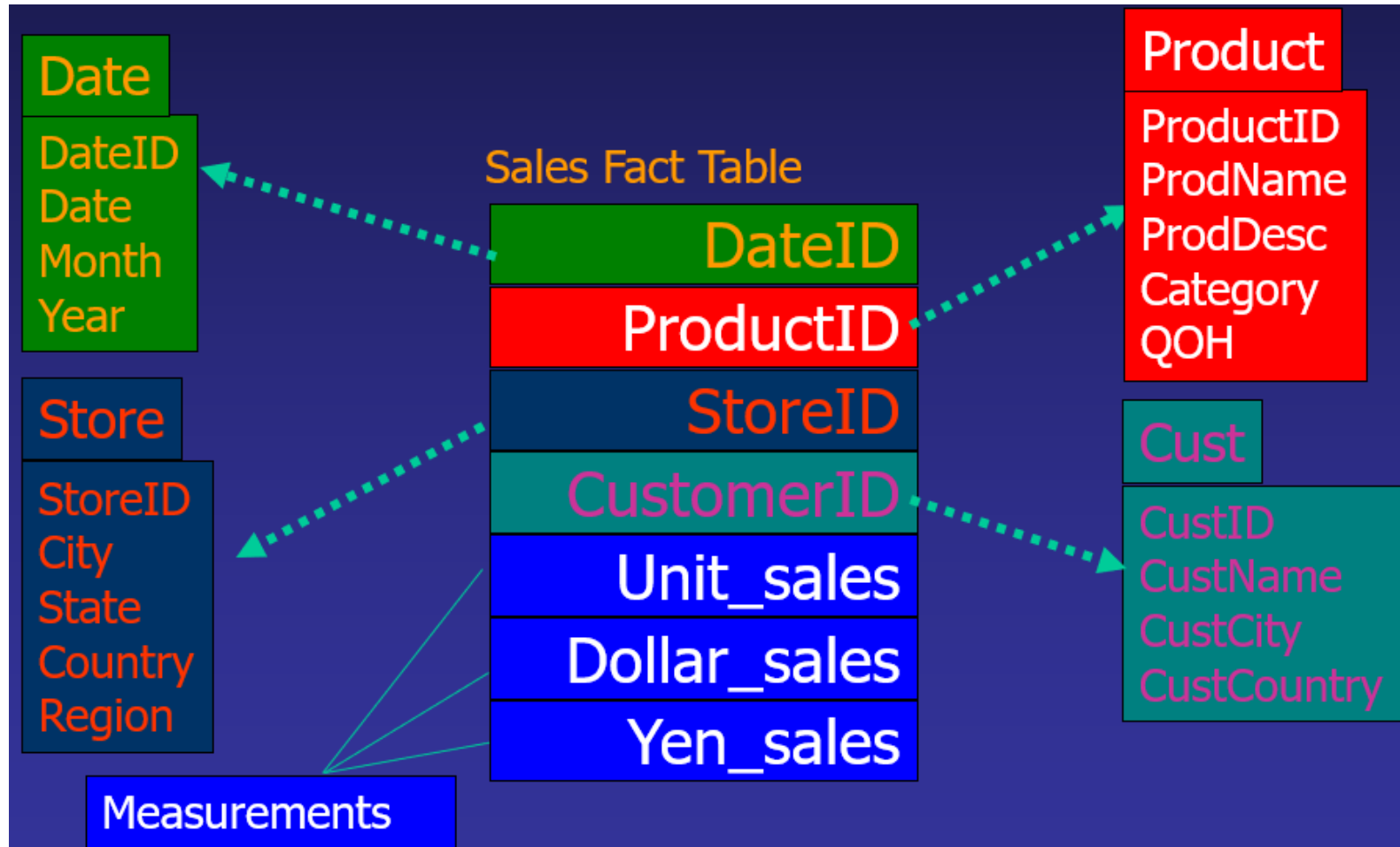
~cont.

■ 示例1:



~cont.

■ 示例2:



~cont.

■ 示例3:

计费系统

cdr	
recd_type	SHORT
num	VARCHAR(15)
dist_code	VARCHAR(4)
imis	VARCHAR(15)
target_num	VARCHAR(24)
msc_id	VARCHAR(10)
lac	VARCHAR(5)
start_time	DATETIME
duration	LONG
serv_type	VARCHAR(2)
serv_code	VARCHAR(4)
data_serv	VARCHAR(6)
data_mark	VARCHAR(6)
chag_flag	VARCHAR(1)
msrn	VARCHAR(12)
norm_chag	DOUBLE
dist_chag	DOUBLE
dist_exa	DOUBLE
prod_time	VARCHAR(8)
bill_time	DATETIME
bill_flag	SHORT
change_flag	SHORT
free_mark	SHORT

营业和帐务系统

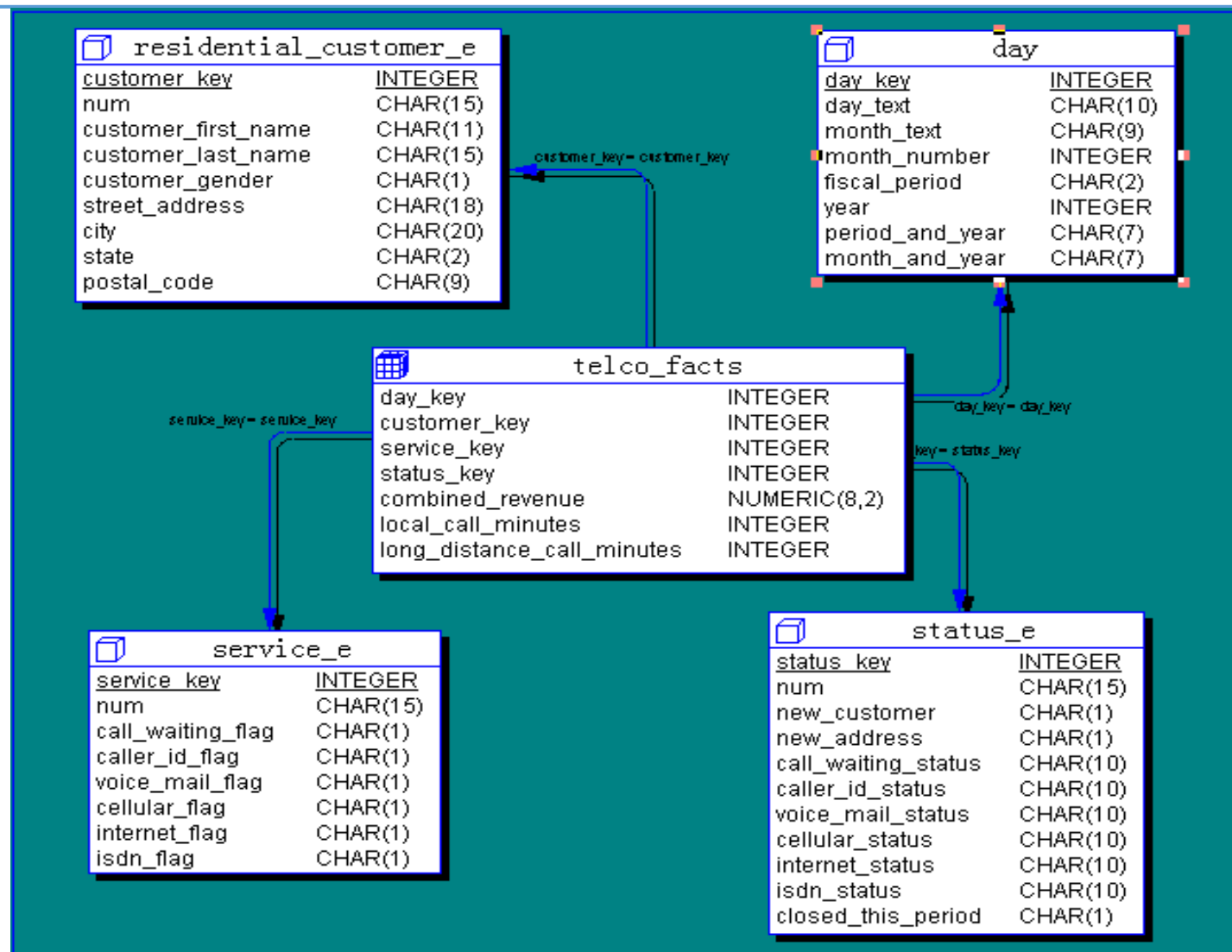
residential_customer	
mobile_num	CHAR(15)
customer_first_name	CHAR(11)
customer_last_name	CHAR(15)
customer_gender	CHAR(1)
street_address	CHAR(18)
city	CHAR(20)
state	CHAR(2)
postal_code	CHAR(9)

service	
mobile_num	CHAR(15)
call_waiting_flag	CHAR(1)
caller_id_flag	CHAR(1)
voice_mail_flag	CHAR(1)
cellular_flag	CHAR(1)
internet_flag	CHAR(1)
isdn_flag	CHAR(1)

status	
mobile_num	CHAR(15)
new_customer	CHAR(1)
new_address	CHAR(1)
call_waiting_status	CHAR(10)
caller_id_status	CHAR(10)
voice_mail_status	CHAR(10)
cellular_status	CHAR(10)
internet_status	CHAR(10)
isdn_status	CHAR(10)
closed_this_period	CHAR(1)

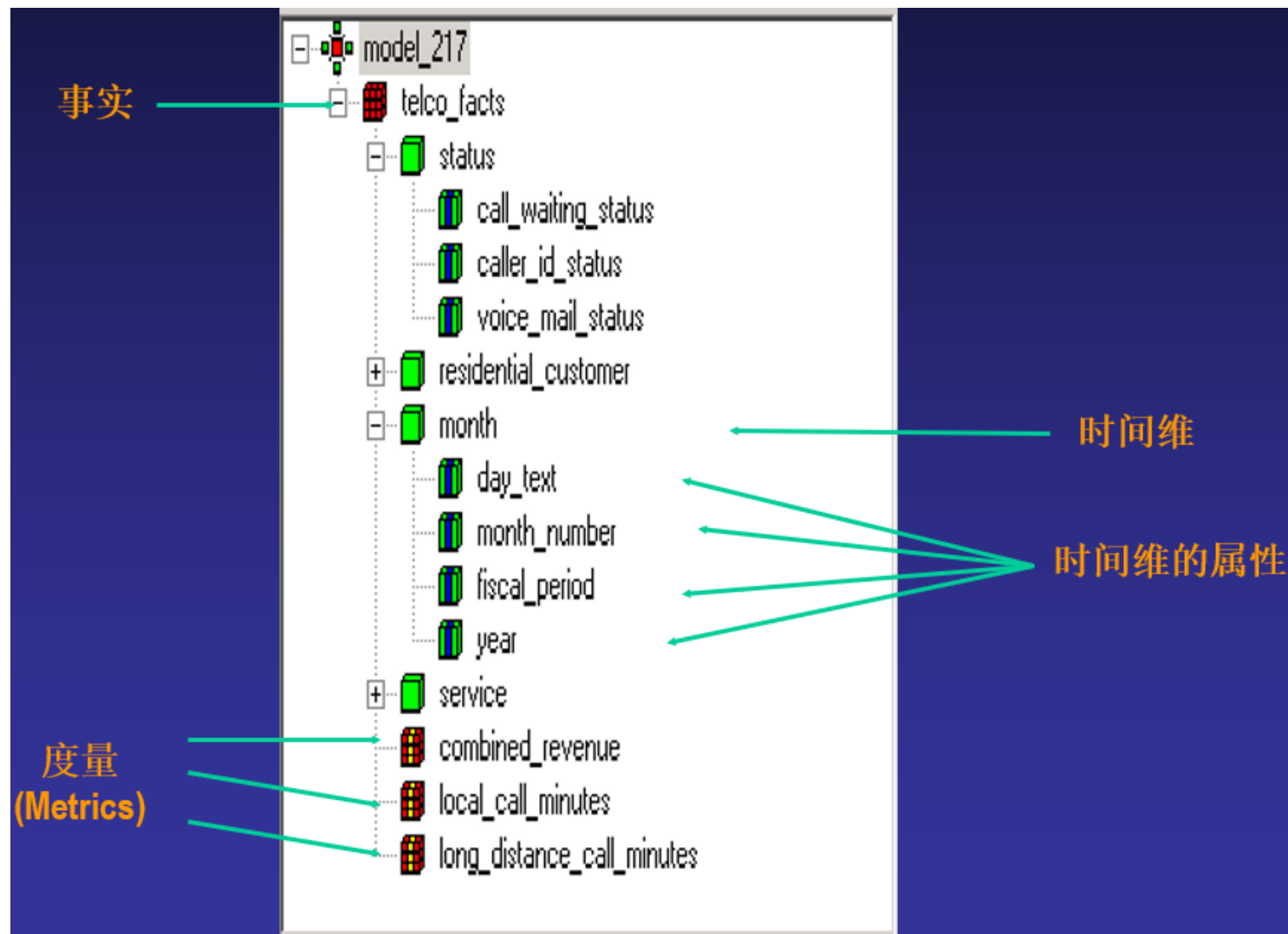
~cont.

■ 示例3:



~cont.

■ 多维模型:

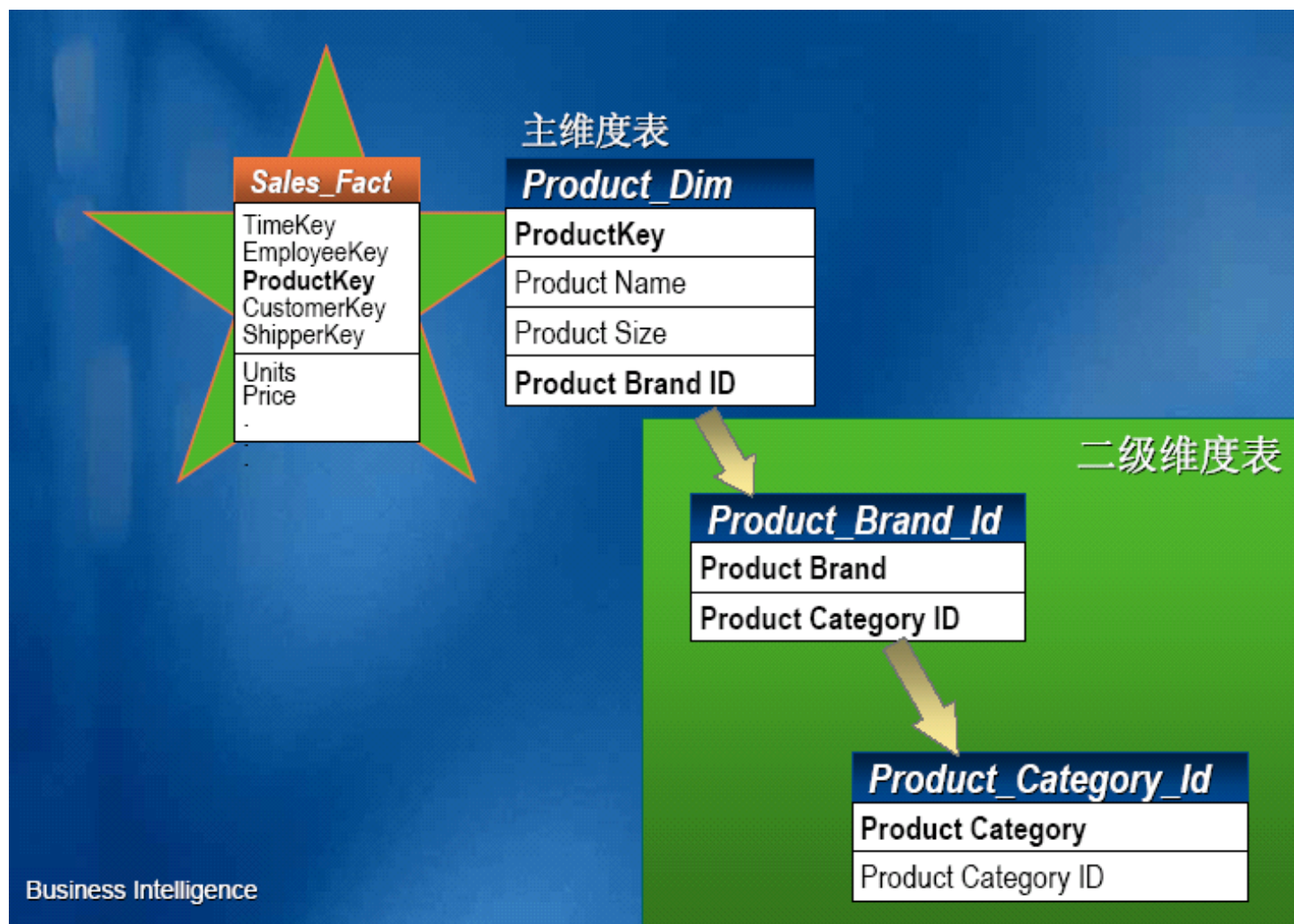


4.2 雪花模型

- 雪花模型的特点：
 - 维表进一步层次化，改善查询性能
 - 增加表数量，增加查询复杂性
 - 节省存储空间
 - 层次性好
 - 易维护
 - 不同于星型模型的反规范化denormalized，雪花模型是规范化的

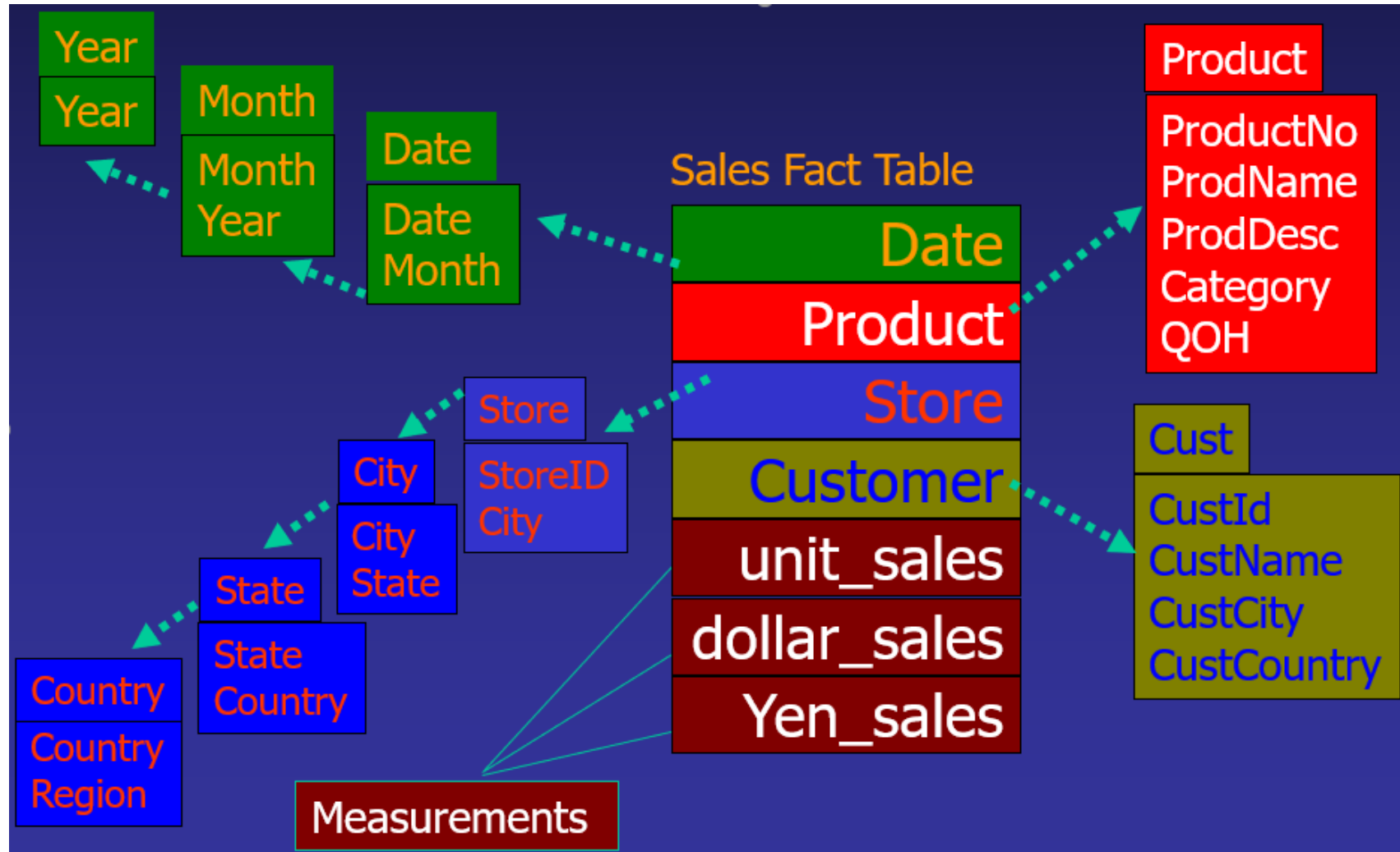
~cont.

■ 示例1:



~cont.

■ 示例2:



~cont.

■ 将星型模型转换为雪花模型的示例

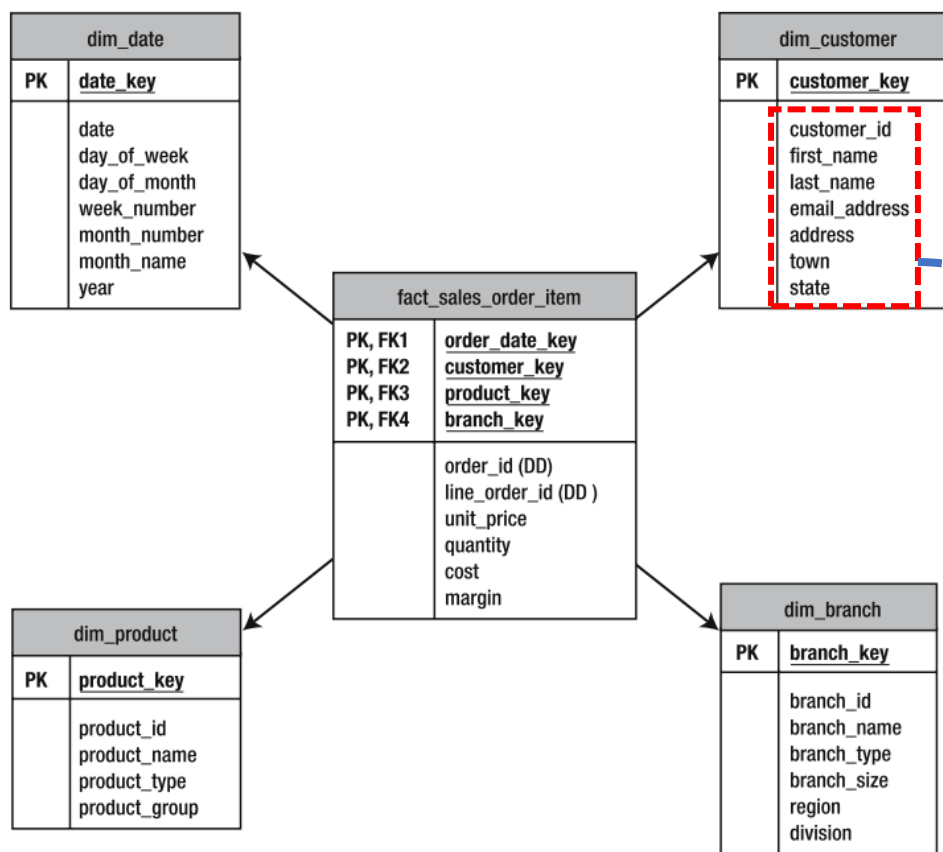


Figure 1-4. Star schema dimensional data store

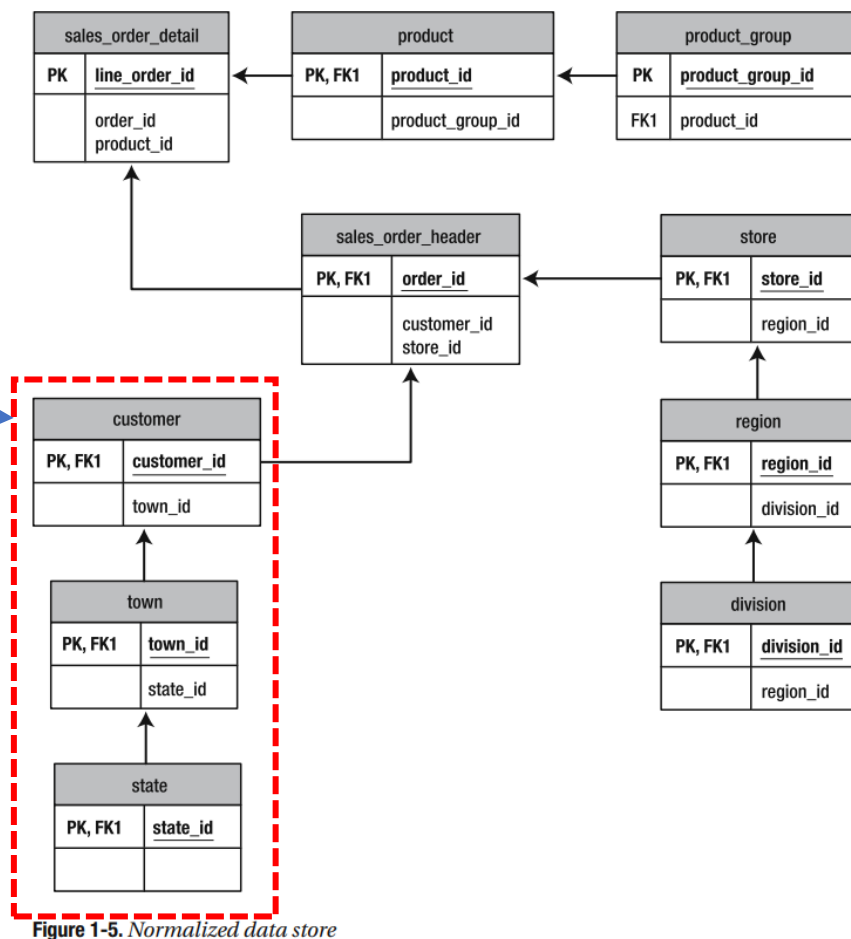
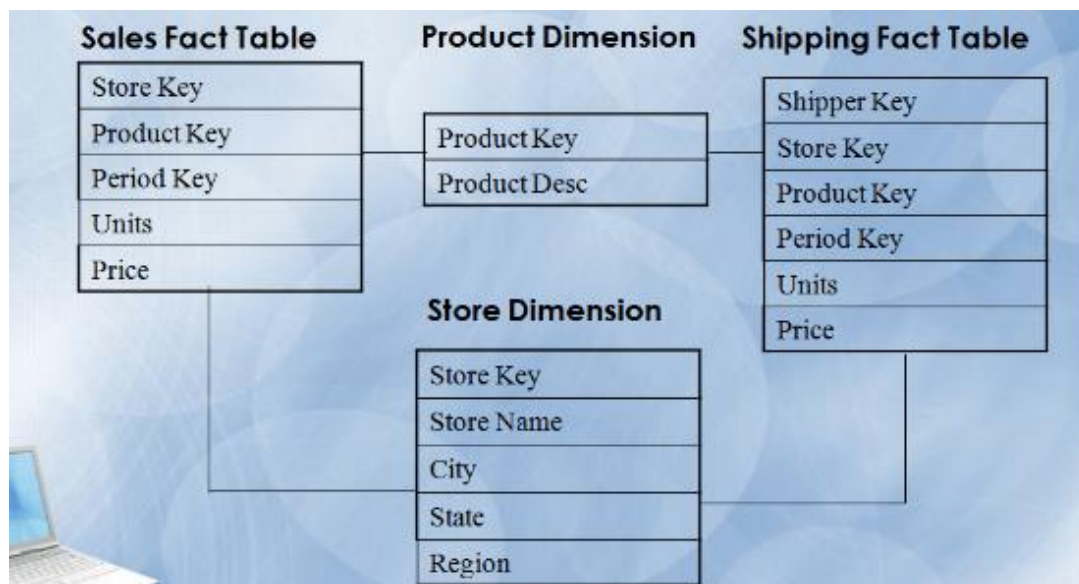


Figure 1-5. Normalized data store

4.3 星座模型

- 星座模型也叫**事实星座模型**(Fact constellation schema)，是对星型模型的扩展，即多个星型模型的集合
 - 两个或以上的事实表共享一个或多个维度表
- 它适用于多个事实表共享一个或多个维度表的复杂查询场景



4.4 三种模型的选择

■ 三种模型比较

- 星型模型通过预连接和建立有选择的数据冗余，为用户访问和分析过程大大简化了数据
- 星型模型效率比较高，因为雪花模型维表层次多，查询的时候连接操作较多
- 雪花模型通过最大限度的减少数据存储量以及联合较小的维表来改善查询性能
- 雪花模型增加了用户必须处理的表数量，增加了某些查询的复杂性，但这种方式可以使系统进一步专业化和实用化，同时降低了系统的通用程度
- 雪花模型的维表可能是规范化形式，以便减少冗余，易于维护，节省存储空间
- 星座模型对复杂查询可能更灵活，但结构复杂，难实现和维护

■ 选择建议：

- 星型模型结构效率上优于雪花模型，首选星型
- 如果存储空间上存在瓶颈，可以考虑使用雪花模型
- 如果维护方面要求简便性，可以考虑使用雪花模型

维度表设计参考：<https://juejin.cn/post/7517029959388414002>

4.5几个典型的数据仓库应用模型

■ 产品销售

— 事实表：销售记录

- 销售额
- 成本
- 产品件数

— 维度表

- 时间
- 产品
- 销售人员
- 客户
- 商店
- 促销

~cont.

■ 网站分析

— 事实表：点击记录

- 人次数
- 人数

— 维度表

- 时间
- 网站
- IP（地域）
- 浏览器

~cont.

■ 电信行业

- 事实表：打电话的记录

- 通话时间
- 本地费
- 长途费

- 维度表

- 时间
- 客户年龄
- 客户信息
- 手机型号
- 电信服务类型

~cont.

■ 财务/预算分析

— 事实表：财务历史

- 实际费用
- 预算费用

— 维度表

- 时间
- 部门
- 科目

4.6粒度的选择

■ 数据粒度级别

- 原因：数据仓库中数据量不同，需要解决的问题多种多样（**细节问题、综合问题**）
- 因此，不同的问题采用不同的数据粒度级别。在数据量较小的环境下，可以采用单一的数据粒度；对于大数据量，需要采用双重或多重粒度

■ 单一粒度

- 直接存储细节数据并定期在细节数据基础上进行数据综合
- 从数据装载之后，所有细节数据都将保留在数据仓库中
- 存储期限（**5~10年**）到了之后，才会导到后备设备（如磁带）中

~cont.

■ 双重粒度

- 对于细节数据只保留近期的数据在数据仓库中，当保留周期到达时，将距离当前较远的数据导出到磁盘上，从而为新的数据腾出空间
- 数据仓库中只保留在细节数据保留周期内的数据，对于这个周期之后的信息，数据仓库只保留其综合数据
- 两者的区别
 - 细节数据在数据仓库的高速存储设备中存储的时间长短不同

~cont.

■ 影响粒度层次划分的几个因素：

— 要接受的分析类型

- 粒度层次越高，就越不能进行细节分析
- 如最低粒度层次定义为月份时，就不能进行按日汇总信息分析

— 可接受的最低粒度

- 粒度划分策略一定要保证数据的粒度确实能够满足用户的决策分析需要

— 能存储数据的存储容量

- 若存储容量有限，则只能采用较高粒度的数据粒度划分策略

~cont.

■ 粒度划分的步骤:

— 第一步: 粗略估算

- 即估算数据仓库中数据的行数和所需的DASD(Direct Access Storage Device, 直接存取存储设备)数, 然后统计1年和5年内可能出现的数据行数
- 一般情况下
 - 数据空间 = 数据量估算 * 2
 - 索引空间 = 数据空间 * 0.5
 - 估算数据量时要考虑:
 - 现有数据日, 月, 季和年的增长率
 - 数据仓库中保留多长时间的历史数据

注意:

- DW系统和OLTP系统不同, 它需要更大的临时空间, 用于排序操作和数据求和汇总
- DW系统也需要较大的缓存空间, 用于驻留经常被访问的数据

~cont.

■ 粒度划分的步骤：

— 第二步：确定粒度的级别

- 根据估算出的数据行和DASD，决定是否要划分粒度以及如何划分

粒度的阈值			
一年期		五年期	
10 000 000	双重粒度级且认真设计	20 000 000	双重粒度级且认真设计
1 000 000	双重粒度级	10 000 000	双重粒度级
100 000	认真设计	1 000 000	认真设计
10 000	实际上任何设计都行	100 000	实际上任何设计都行

~cont.

■ 粒度划分的步骤：

— 第二步：确定粒度的级别

- 根据估算出的数据行和DASD，决定是否要划分粒度以及如何划分
- **一般方法是**：利用常识，建立数据仓库的一小部分，并让用户去访问这些数据。然后根据用户的反馈意见适当调整粒度的级别
- **最坏的想法是**：想要事先设计好所有的粒度级别，再进行数据仓库的建造
- **经验规则**：在第一次的设计周期中，如果50%的工作是正确的，那么整个设计就是成功的

4.7 确定数据分割策略

■ 数据分割考虑的因素：

— 数据量的大小

- 数据量较小，可以不进行分割，或只用单一标准进行分割
- 数据量很大，应当采用多重标准的组合来比较细致地分割数据

— 数据分析处理的实际情况

- 数据分割是跟数据分析处理的对象紧密联系的

— 简单易行

- 选择用于数据分割的标准应当是自然的、易于实施的

— 与粒度的划分策略相统一

- 同一粒度层次上的数据需要进行分割时，应当按照划分粒度层次时使用的标准进行分割

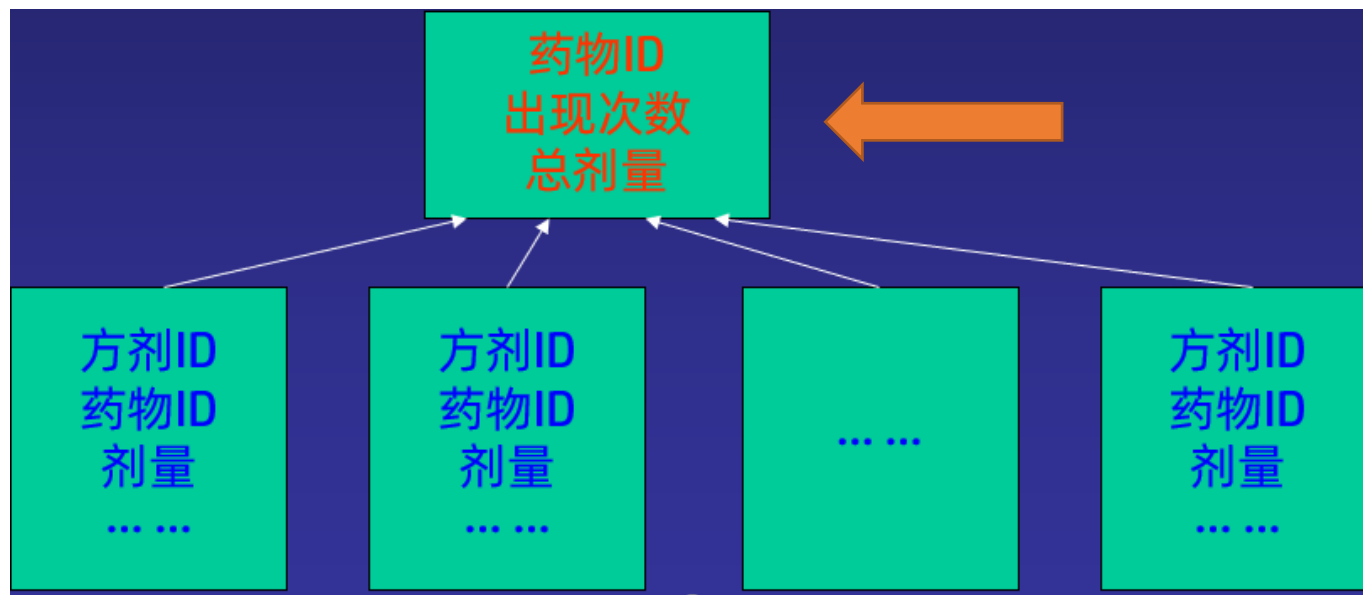
— 数据的稳定性

- 数据仓库中的数据追加频率不同，有的快，有的慢，将不同变化频度的数据放在不同的表中进行更新处理

4.8增加导出字段

- 定义：

- 导出字段是在原始数据的基础上进行总结或计算而生成的数据
- 这些数据可以在以后的应用中直接利用，避免了重复计算



4.9 维度建模中的一些其他关键问题

- 代理键Surrogate key VS. 自然键natural key
 - 数据仓库中人为生成的、与业务含义无关的唯一标识符（通常是表的行号，取自增整数），用于替代业务系统中的自然键
- 一致性维度conformed dimensions
- 参考：
 - 大数据多维分析：维度建模的黄金法则
 - https://blog.csdn.net/m0_62554628/article/details/150397869
 - 大数据治理域——维度表设计
 - <https://juejin.cn/post/7517029959388414002>

5.数据仓库的物理模型

■ 存储结构

- 分布式存储结构
- 集中式存储结构
- RAID磁盘阵列

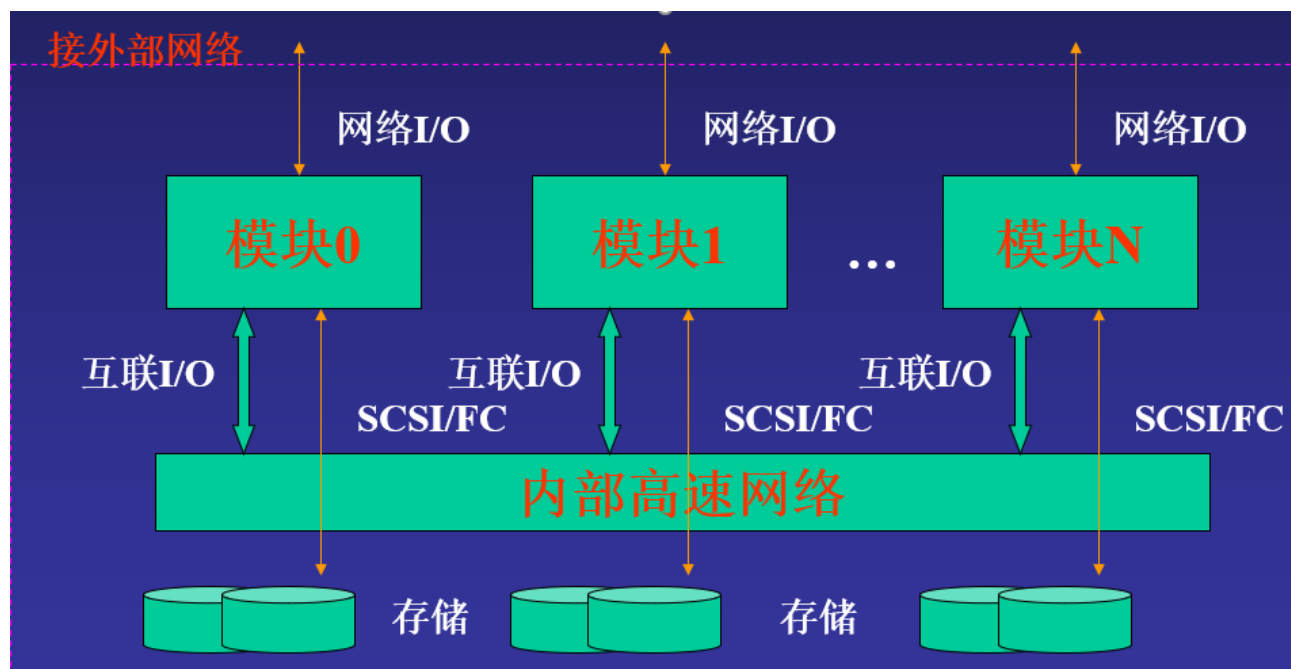
■ 索引

- B树索引
- 位图索引
- 广义索引
- 连接索引

■ 存储策略

5.1 存储结构

- 分布式存储结构
 - 采用磁盘阵列在多个节点间分布的方式来存储数据
 - 物理上是分布的，但是逻辑上是统一的

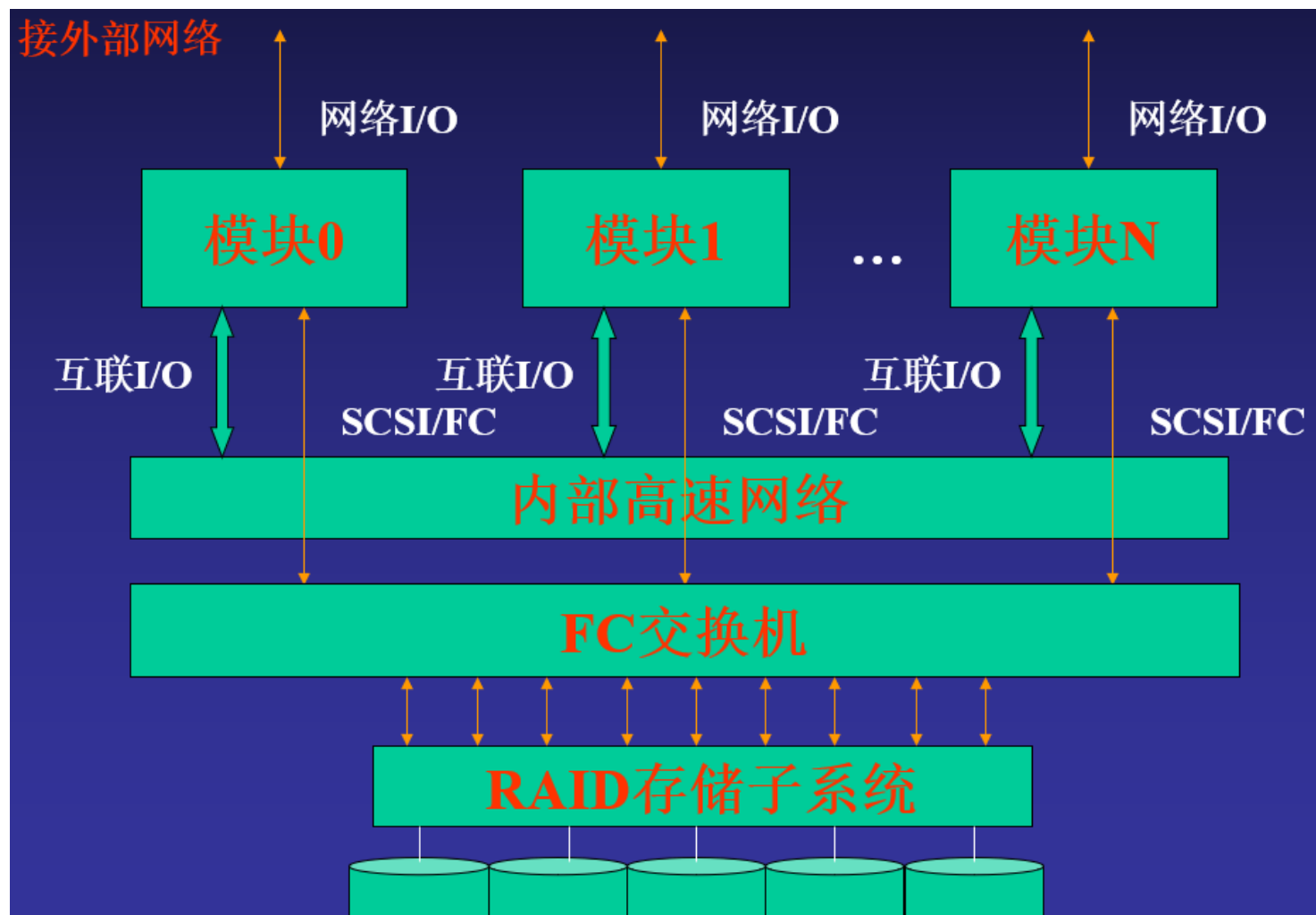


SCSI (Small Computer System Interface) | FC (Fibre Channel)

~cont.

- 集中式存储结构
 - 将现有SAN（Storage Area Network，存储区域网）或者NAS（Network Attached Storage，网络附加存储）作为服务器的存储部分
 - 直接通过FC交换机来直接访问所有的数据而不需要通过其他节点
 - 可以将节点从数据存储管理的负担中解脱出来，实现数据处理和数据存储的分离

~cont.



~cont.

■ RAID磁盘阵列

- 独立冗余磁盘阵列 (Redundant Array of Independent Disks)
- 是一种采用多磁盘驱动器来存储数据的数据存储系统
- 分为6个级别：RAID0、1、2、3、4、5及RAID0+1。RAID2、3、4使用频率较低，RAID5、0、1、0+1使用较多
- 在 RAID 的各个级别中，RAID 1 和 RAID 0+1 提供最佳的数据保护和最佳性能，但就所需的磁盘而言会需要更多的成本。当硬盘成本不是限制因素时，就兼顾性能和容错而言，RAID 1 或 RAID 0+1 是最佳选择。
- RAID 5 的成本比 RAID 1 或 RAID 0+1 低，但是它提供的容错和写入性能较差。RAID 5 的写入性能大约只是 RAID 1 或 RAID 0+1 的一半，这是因为 RAID 5 读取和写入奇偶校验信息需要额外的 I/O

~cont.

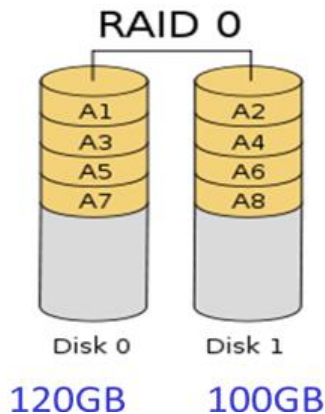
■ RAID可通过并行提高性能

- 通过对若干磁盘的并行存取，可以提高数据的传输速率。可以把数据拆分存储到多个磁盘上，然后进行并行存取
- **比特级拆分**：把每个字节按比特分开，存储到多个磁盘上。如有一个由8个磁盘组成的阵列，将每个字节的第*i*个比特位写到第*i*个磁盘上，它的传输速率将是通常速率的8倍。
 - 推广：使用由四个磁盘组成的阵列，每个字节的第*i*个比特位和第4+*i*个比特位写到第*i*个磁盘上
- **块级拆分**：把整个磁盘阵列看成一个单独的大磁盘，给块逻辑编号。如，对*n*个磁盘的阵列，将阵列逻辑上的第*i*个块存储到第 $(i \bmod n) + 1$ 个磁盘上
- **块级拆分**是最常用的数据拆分形式
- 磁盘系统中的并行有两个主要目的：
 - **负载均衡**多个小的访问操作（块访问），以提高这种访问操作的吞吐量
 - 并行执行大的访问操作，以减少大的访问操作的反应时间

~cont.

❑ RAID0 (non-redundant striping)

- ❶ RAID0把数据拆分到多块磁盘并行存贮。
- ❷ 采用位级拆分且没有任何冗余（如镜像或奇偶校验）。
- ❸ 通常以并行方式工作，可以使数据传输达到单磁盘的几倍。
 - 组成RAID0的单磁盘驱动器数量越多，数据传输速度会越快。
- ❹ 不维护任何冗余信息。
- ❺ 具有最好的读写性能，但安全性最低。
 - 只要有一块硬盘坏，就会造成全部数据因不完整而无法使用。



- 总容量取决于单个磁盘的最小容量。
如，大小=2*min（120GB，100GB）=200GB

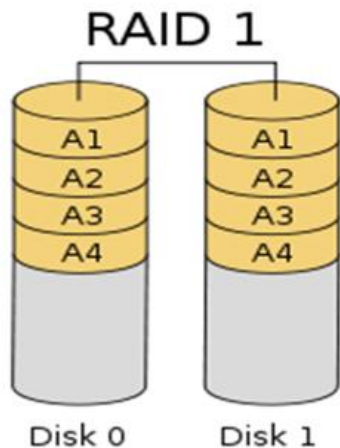
- 故障率

$$\mathbb{P}(\text{at least one fails}) = 1 - \mathbb{P}(\text{neither fails}) = 1 - (1 - 0.05)^2 = 0.0975 = 9.75\%$$

~cont.

❑ RAID1 (mirrored disks)

- 🔗 指的是使用块级拆分的磁盘镜像。
 - 为每一个磁盘配置一个镜像磁盘。镜像磁盘随时保持与原磁盘相同的内容，实现数据的自动备份。
 - 一旦第一块磁盘损坏，互为镜像的第二块磁盘将自动接管工作。
- 🔗 特别适合安全性要求很高场合。
- 🔗 缺点：有效容量利用率只有50%，成本较高。

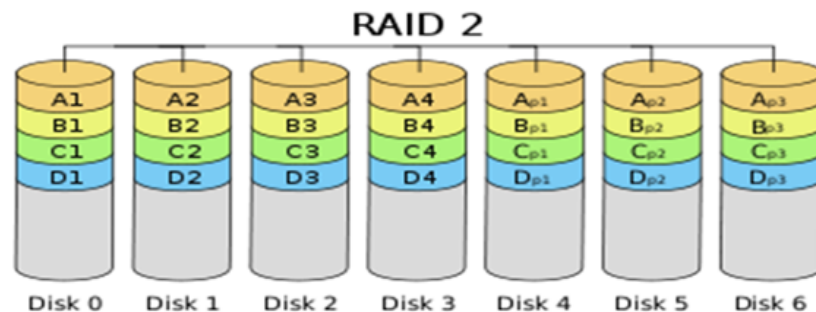


- **注意：**一些厂商使用RAID 1+0级或RAID 10级来指代使用拆分的镜像，使用RAID 1级指代不使用拆分的镜像。
- 不使用拆分的镜像可以与磁盘阵列一起使用，以呈现为一个单一的大的可靠磁盘：如果每个磁盘有M块，逻辑块1到M存储在磁盘1上，逻辑块M+1到2M存储在第二磁盘上，以此类推，每个磁盘都有镜像。

~cont.

❑ RAID2 (error-Correcting Codes错误-校正码)

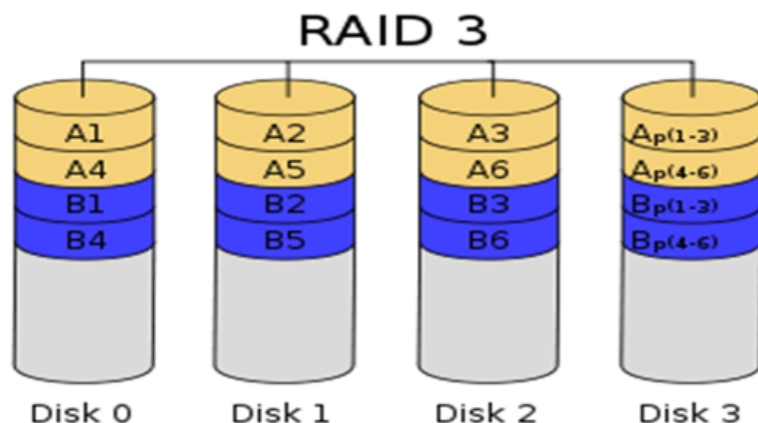
- ❶ 采用若干磁盘（数据盘）拆分存储字节中的位（bits），并对每个字节计算奇偶校验位，额外的校验位存储在冗余盘。如果其中一个磁盘坏，可通过其他磁盘**对应位**与**冗余位**来恢复重建坏盘中的磁盘。
 - 冗余盘除了存储校验数据外，还兼任辅助检测坏盘功能。因此冗余盘往往比较多，如：4个数据盘配置3个冗余盘。
 - 基于位级拆存意味着对有D个数据盘的磁盘阵列中，一次读写传输最少是D个块。因此较有利于传输数据量大的磁盘请求，不利于传输数据量小的磁盘请求。
 - 写操作需要一个read-modify-write的周期过程。



~cont.

❑ RAID3 (Bit-Interleaved Parity位-奇偶交替)

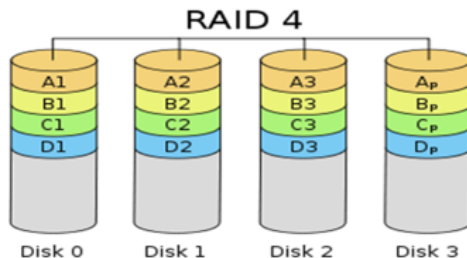
- ❶ RAID2中因配置了较多的冗余校验盘，能自动解决坏盘检测问题，但也增大了代价。实际上，冗余信息只有足够恢复坏盘信息就足够了，因为磁盘控制器本身就可以轻易实现坏盘检测。
- ❷ 与RAID2不同，RAID3只使用一个冗余磁盘，即采用最低的安全性开销。
- ❸ RAID3性能类似于RAID2，读写最小传输单位是D个块。写操作都需要一个read-modify-write 的周期过程。
- ❹ RAID3在实际中使用非常少。这是因为RAID3不能满足并发请求服务。



~cont.

❑ RAID4 (block-Interleaved Parity块-奇偶交替)

- ❶ RAID4拆存单位是一个磁盘块。块级分存优点是能充分利用块设备工作特性，且能适应各种数据量传输的磁盘请求。
- ❷ 不论有多少个数据磁盘，RAID4只用一个冗余盘存储各数据盘中的奇偶校验数据。
 - 如果一个磁盘发生故障，可以使用奇偶校验块和其他磁盘上对应的块来恢复发生故障盘上的块。
- ❸ 读取一个块只访问一个磁盘，因此允许其他的请求在其他磁盘上执行。
 - 每个访问操作的数据传输率较低，但可并行地执行多个读操作，产生较高的总I/O传输率。由于所有磁盘可以并行地读，因此读取大量数据的操作有很高的传输率。
- ❹ 写入大量数据的操作也有很高的传输率，因为数据和奇偶校验位可以并行执行。



~cont.

□RAID4 (续) — 示例

🕒 对一个由4块盘构成的RAID4级，假设数据盘1#、盘2#、盘3#在第*i*块、第*j*个字节数据分别为11010100、01110011、00111110。试计算冗余盘第*i*块第*j*个字节上的数据。

解：三个数据盘第*i*块、第*j*个字节中：最高位8分别是1、0、0，有奇数个1，故对应冗余字节的第8位取1；次高位7分别是1、1、0，有偶数个1，故对应冗余字节次高位取0；……最终可得冗余盘第*i*块第*j*个字节数据值为10011001。

🕒 RAID4的磁盘读写过程

- 读块过程：直接读出相应数据盘中的目标块即可。
- 写块过程：除了写目标数据盘外，还要修改冗余盘上对应块数据。即写单个块需要一个read-modify-write的周期过程。该过程只涉及当前数据盘和校验盘，不涉及其他数据盘。在写块之前，需先读出当前数据盘原目标块的内容，并按下述方法计算校验盘对应块的新数据。

校验盘对应块新数据 = (当前数据盘当前块原数据 XOR 当前数据盘当前块新数据)
XOR 校验盘对应块原数据

~cont.

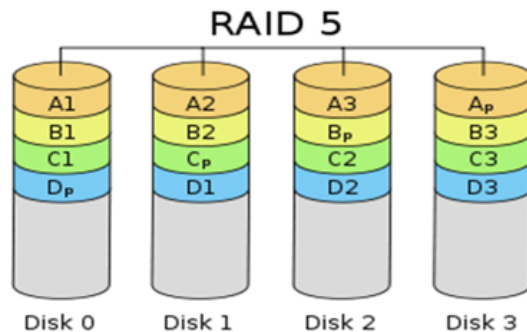
□ RAID5 (block-Interleaved distributed Parity)

🔗 是RAID4的改进。

- RAID4中校验数据块总是用一个固定盘来存储，而在RAID5中，校验块是交替均匀分布在各磁盘上。
- 如，对于一个包含4块盘的阵列，在各盘的第 i 个柱面上，可选用 $(i \bmod 4)$ 号盘的第 i 柱面来存储冗余数据。

🔗 在具有冗余安全性、能适应各种大小传输量的RAID系统技术中，RAID5是性能最好的一种。

- 在有5个盘的阵列中，RAID4和RAID5的磁盘空间利用率都是80%，磁盘空间利用会随着数据盘数的增加而增加。



~cont.

□RAID5 (续) — 示例

- 在5个磁盘组成的阵列中，逻辑块 $4k$ ， $4k+1$ ， $4k+2$ ， $4k+3$ 对应的奇偶校验块 P_k ，存储在第 $(k \bmod 5) + 1$ 个磁盘中；余下的四个磁盘中对应的块存储从 $4k$ 到 $4k+3$ 这四个数据块。下表描述了从0到19这前20个块和它们的奇偶校验块在磁盘中是如何排列的（对于以后的块重复这样的存储模式）。

P0	0	1	2	3
4	P1	5	6	7
8	9	P2	10	11
12	13	14	P3	15
16	17	18	19	P4

Disk 0 Disk 1 Disk2 Disk3 Disk4

- 注意：

- 奇偶校验块不能与其所对应的数据块存储在同一个磁盘上。如果这样，一个磁盘发生故障会导致数据和奇偶校验位的丢失，数据将不可恢复。
- 在相同成本下，RAID5提供比RAID4更好的读写性能，所以RAID4级在实际中不使用。

~cont.

□ RAID级别的选择

④ 应考虑以下因素：

— 所需的额外磁盘存储带来的花费；在I/O操作数量方面的性能需求；磁盘故障时的性能；数据重建（即故障磁盘上的数据在新磁盘上重建的过程）过程中的性能。

④ RAID0级用于数据安全性不是很重要的高性能应用。

④ RAID1级提供最好的写操作性能，广泛应用于数据库系统日志文件的存储这样场合。

④ 对于经常进行读操作而很少进行写操作的应用，RAID5级是首选。

④ RAID2级包含在RAID3级（比特级拆分），RAID4级包含在RAID5级（块级拆分）。

□ 一些硬件RAID实现允许热交换（**hot swapping**，就是在不切断电源的情况下将出错磁盘用新的磁盘替换）

5.2索引

■ B树索引

– 1972年R.Bayer和E.M.McCreight提出了一种称之为B-树的多路平衡查找树。
它适合在磁盘等直接存取设备上组织动态的查找表

– B-树的查找方法

- 在B-树中查找给定关键字的方法类似于二叉排序树上的查找。不同的是在每个结点上确定向下查找的路径不一定是二路而是keynum+1路的
- 对结点内的存放有序关键字序列的向量key[l..keynum] 用顺序查找或折半查找方法查找。若在某结点内找到待查的关键字K，则返回该结点的地址及K在key[1..keynum]中的位置；否则，确定K在某个key[i]和key[i+1]之间结点后，从磁盘中读指针所指的结点继续查找……。直到在某结点中查找成功；或直至找到叶结点且叶结点中的查找仍不成功时，查找过程失败

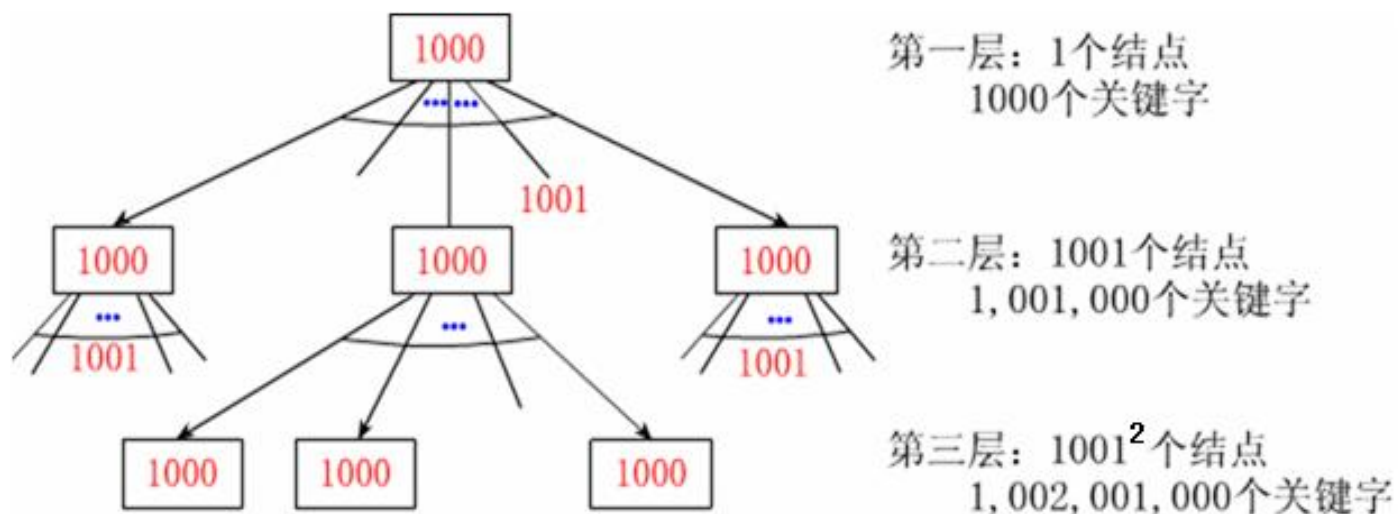
~cont.

■ B树的结点规模

- 在大多数系统中，B-树上的算法执行时间主要由读、写磁盘的次数来决定，每次读写尽可能多的信息可提高算法的执行速度
- B-树中的结点的规模一般是一个磁盘页(page)，而结点中所包含的关键字及其孩子的数目取决于磁盘页的大小
- 注意：
 - 对于磁盘上一棵较大的B-树，通常每个结点拥有的孩子数目(即结点的度数) m 为50至2000不等
 - 一颗度为 m 的B-树称为 m 阶B-树
 - 选取较大的结点度数可降低树的高度，以及减少查找任意关键字所需的磁盘访问次数

~cont.

- 例子：下图给出了一棵高度为3的1001阶B-树



一棵高度为3的1001阶B-树示例

- 每个结点包含1000个关键字，故在第三层上有100多万万个叶结点，这些叶节点可容纳10亿多个关键字
- 图中各结点内的数字表示关键字的数目
- 通常根结点可始终置于主存中，因此在这棵B-树中查找任一关键字至多只需二次访问外存

~cont.

■ B树的使用条件和特点

— 使用条件

- 当要求被索引的字段的值是唯一或者是接近唯一的时候
- 查询工作是从大表中选择少数几个数据记录时

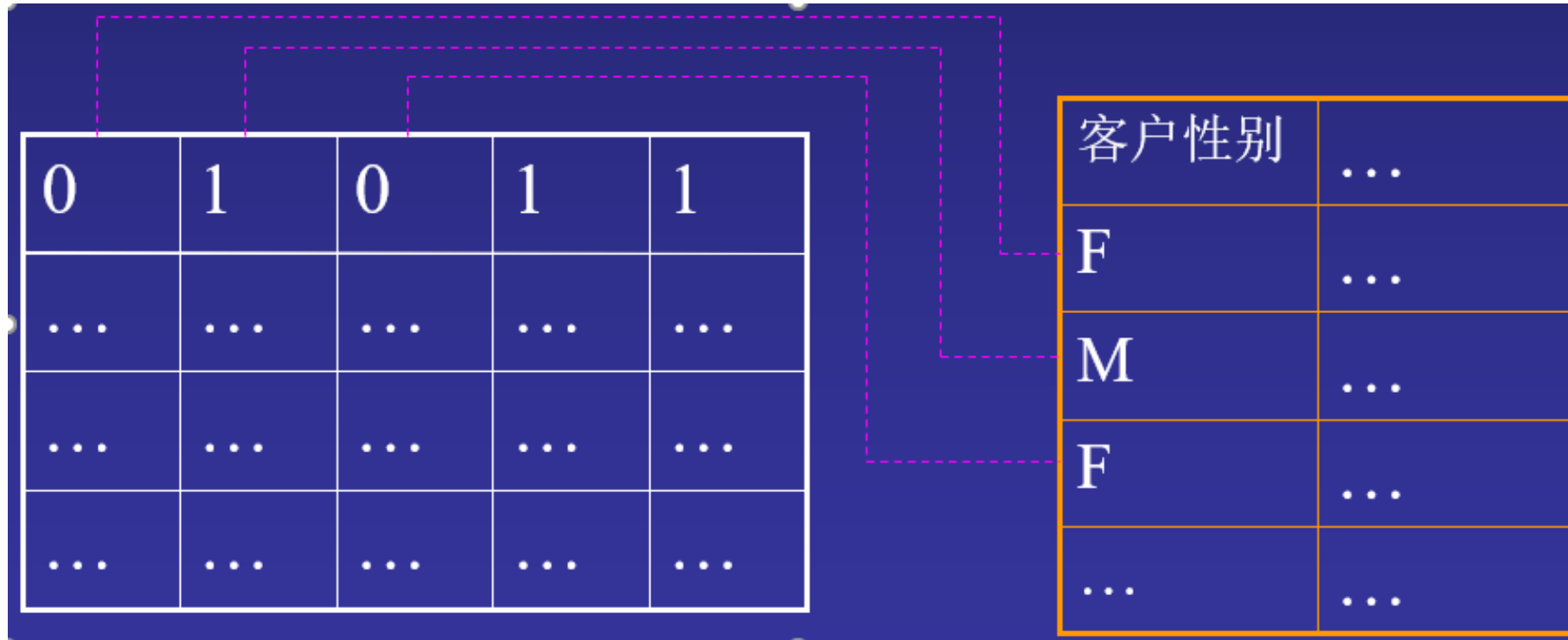
— 特点

- B树快速索引性能是通过较大的代价换取的。通常为一个大表建立B树索引需要大量的时间和磁盘空间
- B树使用期间的维护工作量很大
- 数据仓库中的数据定期追加，就需要做大量的索引工作
- 如果系统的数据10%-15%发生了变化，就需要对B树索引进行重构

~cont.

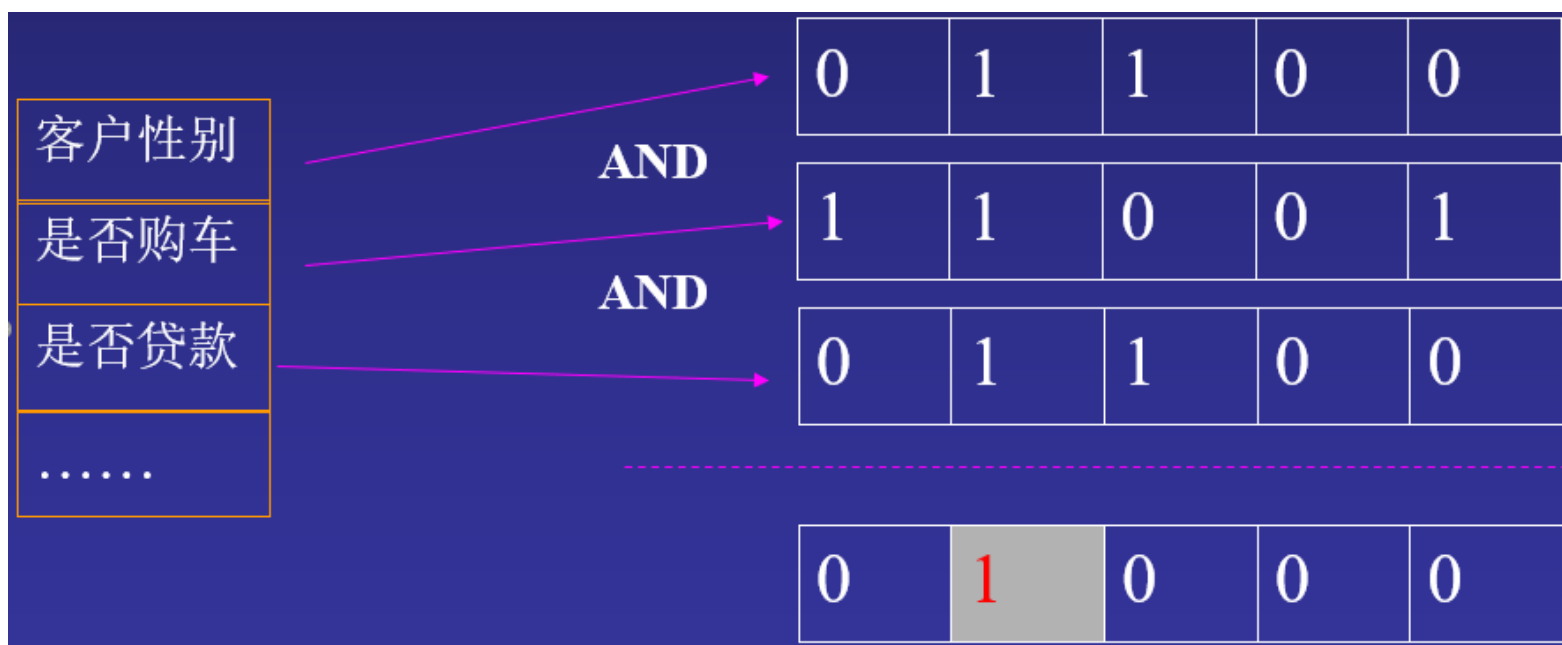
- 位图索引（Bitmap）定义
 - OLAP分析中，需要对大量数据进行**综合性查询**，而不是几条记录，这种情况下B树索引就受到了很大限制
 - 位图索引是一种资源耗费很小、对于多维查询性能很好的索引方法
 - 位图索引是一组0和1字符，表中**每一条记录都被分配一个在位图中的相对位置**且给定一个值（0或者1）
 - 位图中每个比特对应一条记录，所以位图索引占用的空间很小

~cont.



~cont.

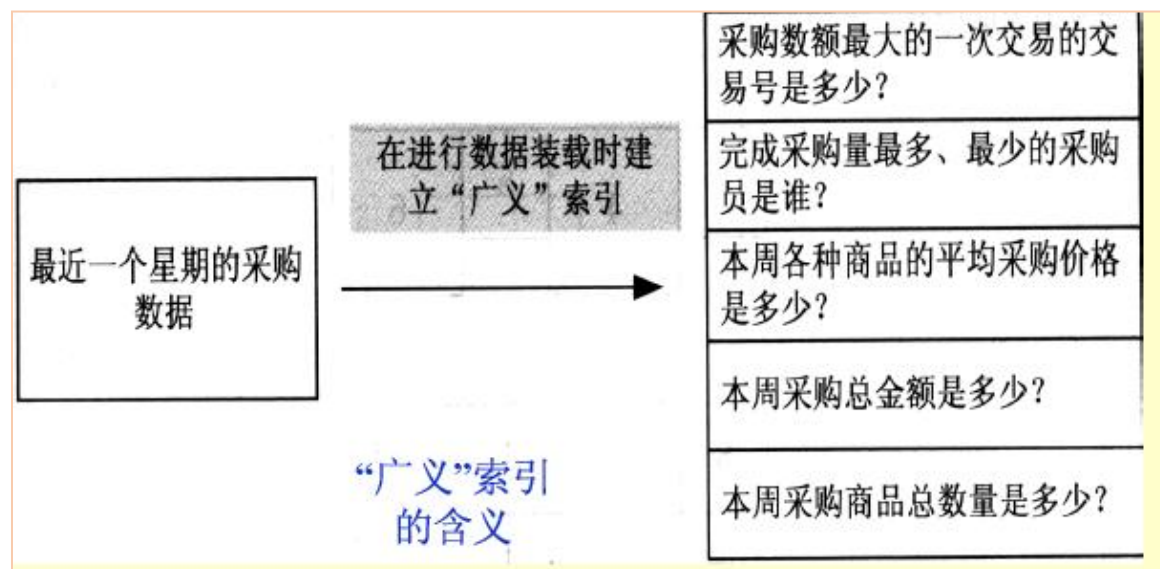
- 在位图上运用**逻辑AND**和**OR**可以非常迅速地得到多维查询结果
 - 例如“**通过贷款购买汽车的男性客户有哪些**”，就可以在商品类型、客户性别、支付方式等字段上建立位图索引，并在查询中综合使用



~cont.

■ 广义索引

- 广义索引是在数据装载的同时建立，它是一种元数据
- 广义索引涉及的是用户最关心的问题，需要在数据加载前进行调查了解，比如使用频率最高的10种药物，使用频率最低的药物等
- 比较**适合经常性**的查询，开销比较小

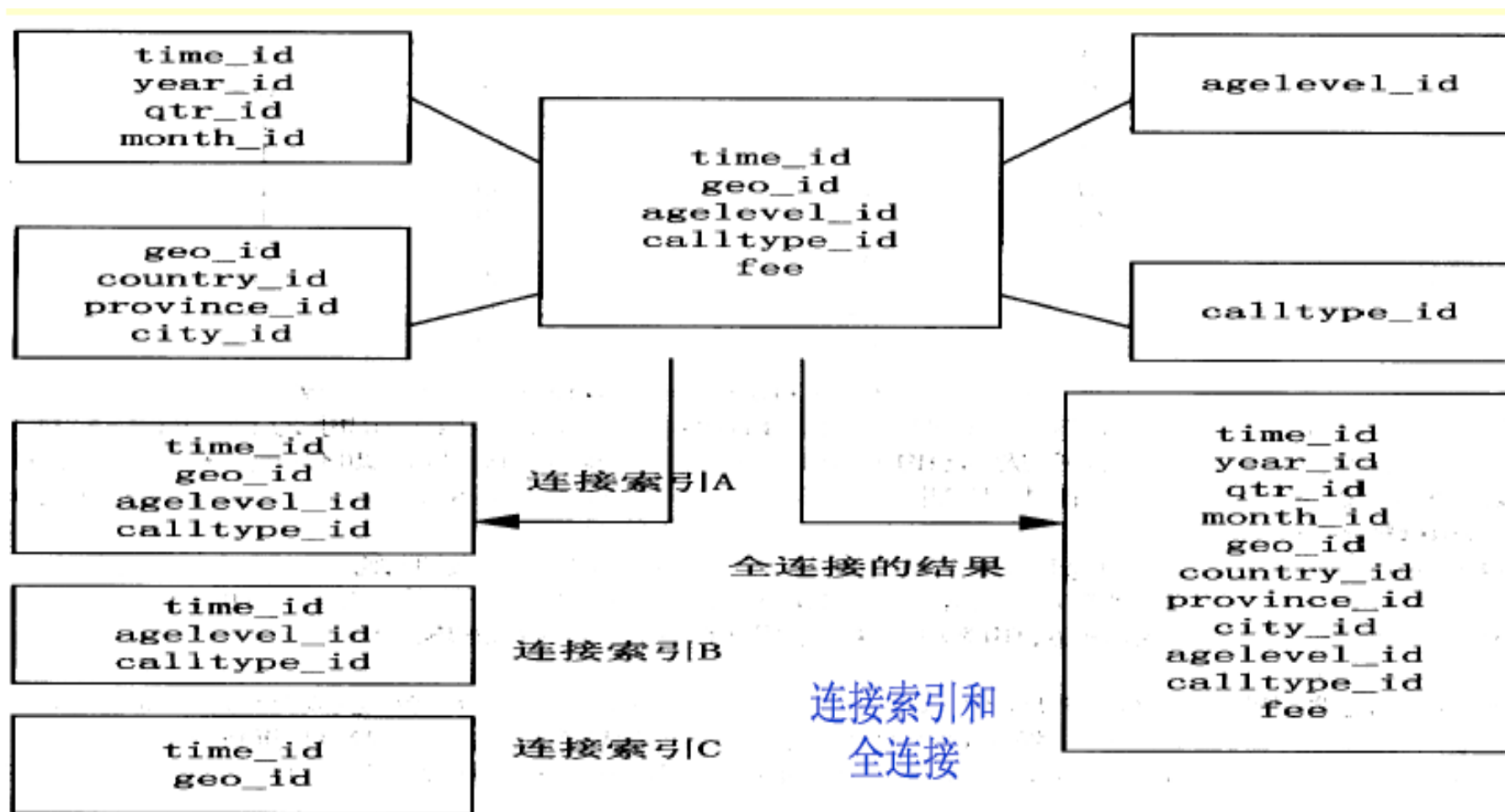


~cont.

■ 连接索引

- 就是将事实表和维表中的索引项进行连接运算，然后将结果作为索引保留
- 如果对事实表和维表中的所有外键进行连接运算，将得到一个很大的表，这种索引称为**全连接索引**
- 但在一般情况下，不一定对所有外键都进行连接运算，可根据需要使用部分外键，生成不同的连接索引
- 在实际应用中，通常**先建立全连接索引**，然后根据需要增加其他的连接索引

~cont.



5.3 数据存储策略

- 数据存储策略

- 考虑因素
- 数据的重要程度
- 粒度
- 使用频率
- 响应时间

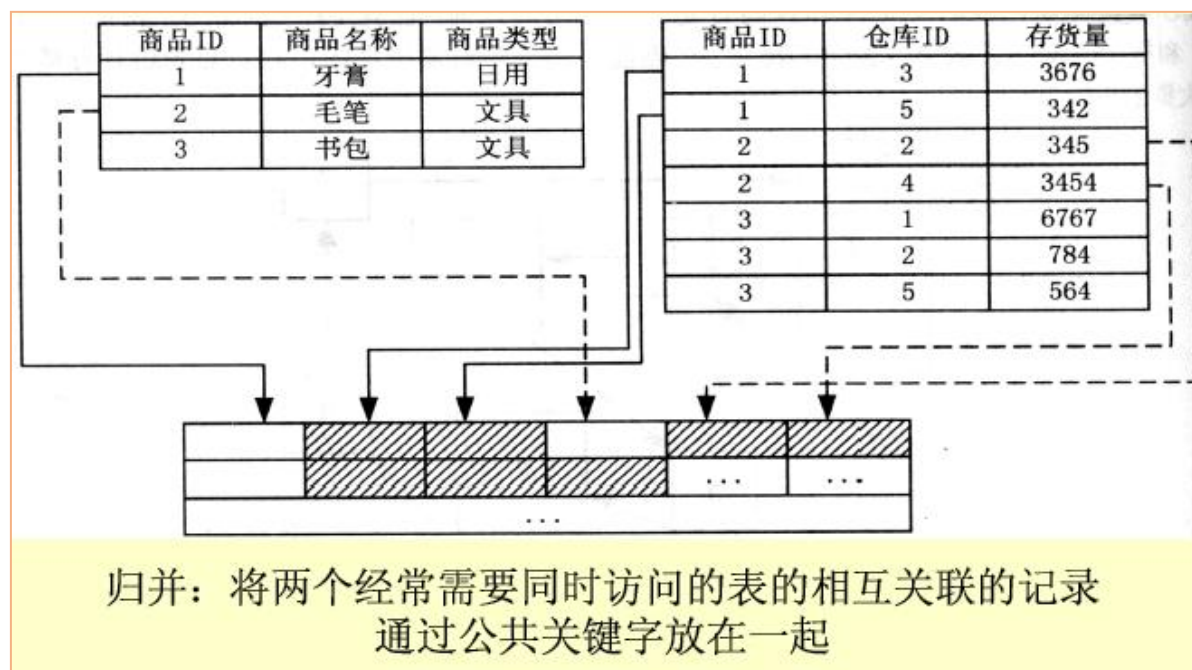
- 采取措施

- 重要程度高、使用频率高或对响应时间要求高的数据存放在高速存储设备上
- 存取频率低或对响应时间要求低的数据存放在低速存储设备上
- 除此之外，还有以下策略：

~cont.

■ 合并表

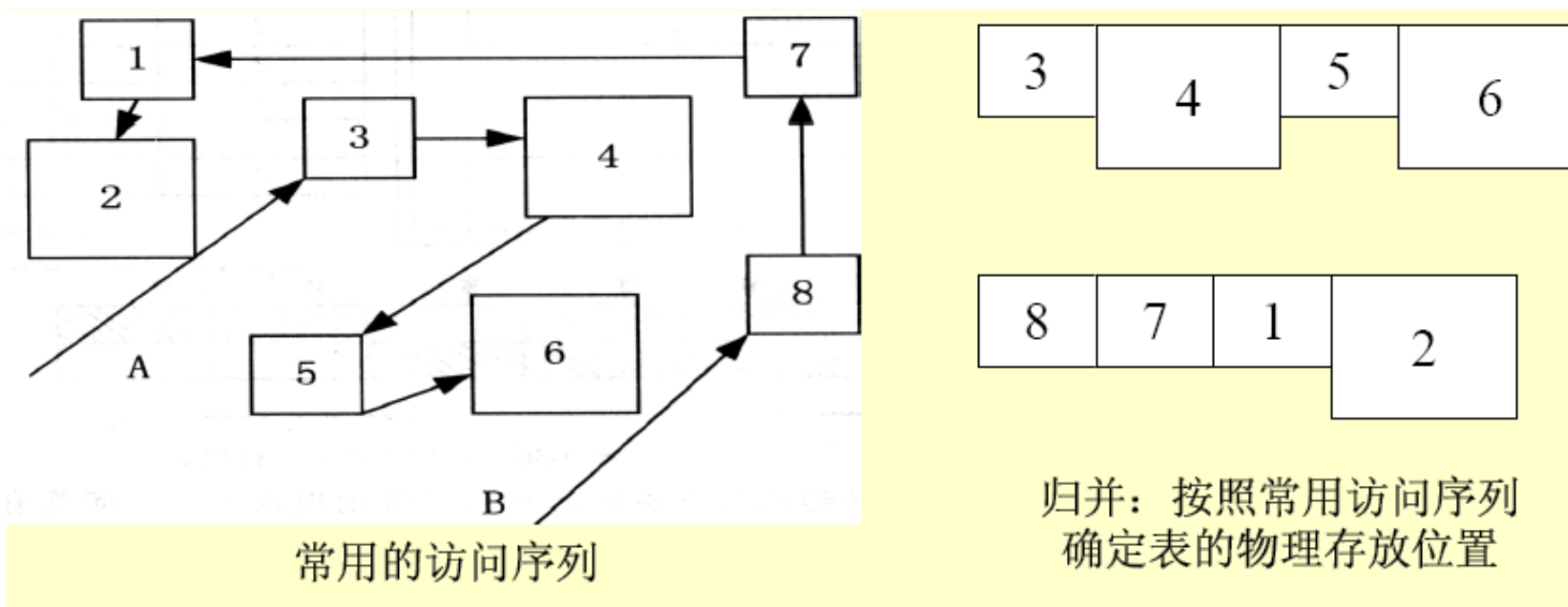
- 例行分析处理时，涉及的表存储时放到一起，可大大减少磁头定位时间，提高I/O效率
- 这种将多个表中相互关联的记录相邻存储的方式称为**合并表**



~cont.

■ 数据序列

- 把经常按照某一序列访问并处理的一组表存储在一起可减少I/O次数



~cont.

■ 按列存储

- 同一列数据相邻存储
- 同一列数据具有相同的数据类型，按列存储有许多优点，如读取方便、索引方便、统计方便等
- Sybase数据仓库产品是按照列来存储的

药物ID	味性	剂量	性温	性平	性寒	...
1	性温	9				
2	性平	8	9	8	5	...
3	性寒	5				
...	...	...				

~cont.

- 分割表存放
 - 在逻辑设计中对大表进行分割，分割成多个小表，提高访问效率
 - 在物理上采用分布式的存储，达到并行读取的目的

5.4存储分配优化

■ 存储分配优化

- 解决数据分块大小、缓冲区单元大小和个数等问题
- 和系统配置相关，数据仓库厂商会给出推荐配置
- 设计人员在系统维护过程中根据实际情况来调整

■ 设置正确的块大小

- 一个文件包括很多块，每个块包括很多条记录
- 块是数据库的数据和内存中间I/O传输的基本单位
- 块头中存放记录控制信息
- DBMS都有默认块大小，如32kb或者64kb
- 增加块的大小，则可以有更多的记录放入一个块中，一次可以读入更多的记录，减少读操作的次数
- 但当即使只需要大块中很少的记录时，也要读入很多记录，影响了内存管理

~cont.

■ 设置合适的块使用参数

— 块空闲率

- DBMS为每个块预留一部分空间，使块中的记录能够扩展
- 比如块空闲参数设置为20，表示每个块的20%为记录更新修改预留空间
- 当预留空间已满时，DBMS需要找到一个新的扩展区间，在工作中寻找新的区间成为动态扩展，动态扩展带来了巨大的开销
- 分配较大的初始扩展空间可以减少动态扩展
- 数据仓库中，由于更新不多，主要是追加，这个参数设置的越小越好

— 块使用率

- 它指定了一个界限，当块中已使用空间低于这个界限时才允许向该块中插入新的数据行
- 如该参数设置为40，表示当从空间中删除行时，空间不能马上被使用，只有当已经使用的空间降低到40%以下，空闲空间才可以使用
- 数据仓库中，很少删除记录，除非是将数据仓库中的数据存入磁带中，一般情况下，是将这个参数设置的越大越好

6.数据仓库的设计步骤

- 数据仓库设计步骤概述

- 高层设计

- 中层设计

- 低层设计

- 数据仓库生成

6.1 高层设计

- 即概念模式设计
 - 主要考虑商业过程和商业需求的集成，将与目前商业过程有关的信息和数据仓库视图实现的目标合并在一起，创建信息包图
- 主要工作：
 - 确定系统边界：决策类型、需要的信息、原始信息
 - 确定主题域及其内容：主题域的公共键码、主题域之间的联系、属性组
 - 确定维度：如时间维、销售位置维、产品维、组别维等
 - 确定级别：相应维的详细层次（粒度）
 - 确定度量：也称指标和事实，用于进行分析的数值化信息

~cont.

- 创建信息包图：
 - 确定涉及的主题域，例如：时间、顾客、地理位置和产品
 - 设计可以跟踪的，确定一个事件怎样被完成和运行的**关键指标(KPI)**
 - 决定在给定的用户分析或查询中实际包含了多少数据
 - 确定用户如何按**层次聚合数据**和移动数据
 - 决定**数据层次**
 - 估计数据仓库的大小
 - 确定一个数据仓库里数据的**更新频率**
 - 定义如何访问数据
 - 决定数据如何被传递给数据仓库的用户

6.2 中层设计

- 即逻辑结构设计
 - 建立数据仓库的逻辑模型，对前期收集的信息的细化，将信息包图转换成星型模型
- 主要工作：
 - 定义**指标实体**。将信息包图中的内容放到星形图中，每个信息包图代表一个完整的星和信息包。维度实体在指标实体中表示
 - 定义**维度实体**。信息包图中的每一个维度实体位于星形图的星角上，它以维度表的形式存在。随着维度实体在星形图上的定位，进一步定义它与指标实体间的关系
 - 定义详细类别实体。在一个简单的星形图模型中，指标实体被用于访问的维度实体包围。但是在有的决策分析中仅仅知道指标值是不够的，需要定义详细类别实体，将星型图模型转换为雪花图模型
- 设计成果：
 - 适当的粒度划分
 - 合理的数据分割策略
 - 定义适当的数据源
 - 如必要，添加导出字段

6.3 低层设计

- 即物理模型设计
 - 低层设计阶段的任务：建立数据仓库的物理模型，确定数据仓库的存储结构、数据的存储位置和索引策略
- 设计人员需要了解和考虑的问题：
 - 所选用的数据库系统，特别是存储结构和存取方法
 - 数据环境、数据的使用频率、使用方式以及响应时间
 - 存储设备的特性等
- 主要工作：
 - 确定数据的**存储结构**。不同的存储结构有不同的实现方式，应综合考虑存取时间、存取空间利用率和维护代价等因素，根据各种存储结构的优缺点和适用范围选择合适的存储结构
 - 确定数据的**存储位置**。对数据按照其重要程度、使用频率和对响应时间的要求等进行分类，并将不同类别的数据存储在不同的存储设备中。设置存储分配参数，对块的大小、缓冲区的大小和个数等进行物理优化处理
 - 确定**索引策略**。通过对数据存取路径的分析，为各个数据存储建立专用的索引，以获得存取的高效率

6.4数据仓库的生成

- 设计接口
 - 从面向应用和操作的环境生成完整的数据
 - 基于时间的数据转换
 - 数据的聚合
 - 对记录系统进行扫描，以便追加数据
- 数据装入
 - 确定数据装入的顺序
 - 过滤无效或错误数据
 - 数据老化
 - 数据粒度管理
 - 数据刷新

7.数据仓库的建设过程

- 需求分析
- 数据分析
- 技术路线
- 应用分析
- 数据仓库部署
- 数据仓库系统运行与维护

7.1 需求分析

- 数据仓库项目需求分析的特点
 - 需求比较模糊，需求分析过程贯穿整个数据仓库设计过程中
 - 数据仓库开发过程中每一个阶段的每一项任务都是由需求决定的
 - 设计阶段的每一个决策，无论是数据设计、体系结构设计、信息传递方法的安排，都受需求的影响

- 需求分析的内容
 - 常见主题
 - 常见分析问题
 - 数据源的数据库类型
 - 工作平台
 - 数据量及其质量
 - 数据仓库的建立环境
 - 网络状况
 - 各方面各层次人员对数据分析的需求等

~cont.

- 数据仓库项目需求分析文档组成：
 - 绪论。阐述项目的目的和范围，包括项目论证，文档概要
 - 总体需求描述。阐述数据仓库中需要什么样的数据
 - 具体需求。包括源数据的细节，数据转换和存储需求，描述用户需要的各类信息传递方法
 - 指标和维度。包括重要的指标和事实，以及分析这些指标所用的维度
 - 其他需求。数据抽取频率、数据载入方法等
 - 用户期望。说明用户期望如何使用数据仓库
 - 用户参与。列出用户在开发周期中希望参与的任务和活动
 - 实施计划。给出一个合理的数据仓库项目实施计划

7.2 数据分析

- 数据仓库数据分析（即数据仓库设计步骤的内容）
 - － 概念模型设计
 - － 逻辑模型设计
 - － 物理模型设计
 - － 数据装载接口设计

7.3 技术路线

- 技术路线主要内容

- 技术路线主要确定数据仓库的**基础构造**，包括服务器硬件、操作系统、网络软件、数据库软件、局域网和广域网、供应商提供的工具、人员、工作步骤和培训等

- 操作基础构造

- 包括人员、流程、培训和管理软件，是维持数据仓库所需要的，它们支持数据仓库的管理工作并保持它的工作效率

- 物理基础构造

- 计算机平台
 - 工具

~cont.

■ 计算机平台

— 硬件和操作系统

- 硬件：选型时考虑可扩展性、供应商的支持和稳定性
- 操作系统：与硬件相容，选型时考虑可扩展性、安全性、可靠性、内存管理和保护

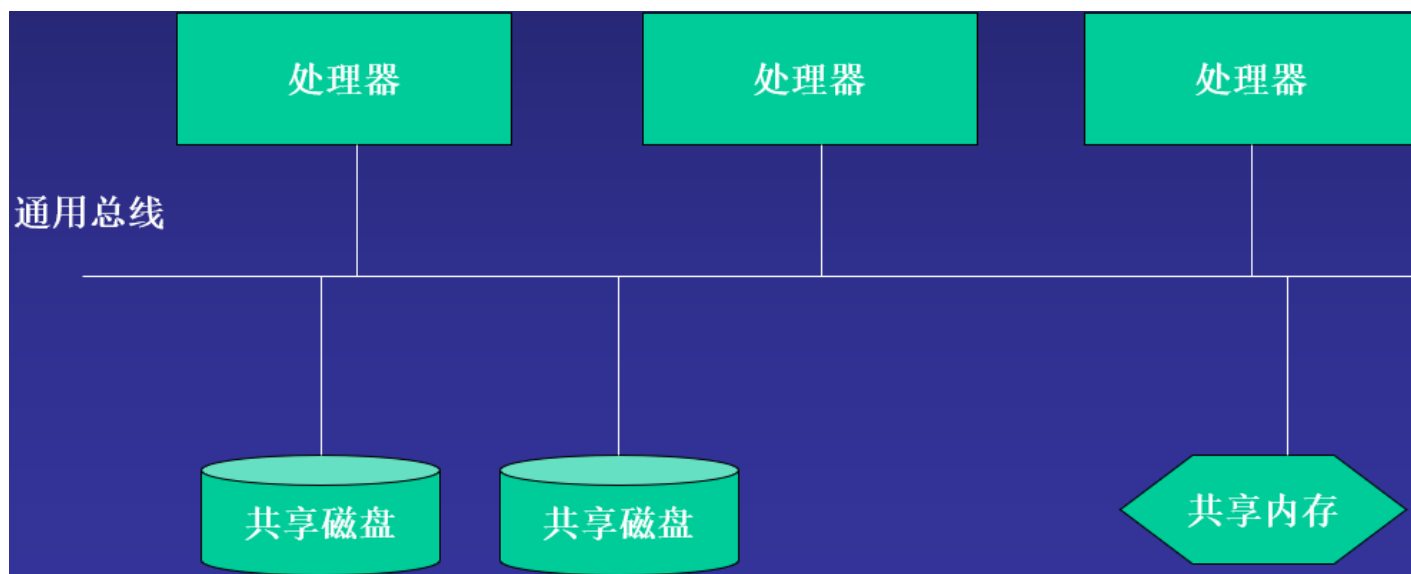
■ 服务器硬件方案

- 数据仓库用户数量和查询数量都会大幅度增加
- 数据载入工作也急剧增多
- 可扩展性和查询性能优化是服务器硬件选择的两个关键因素
- 主要的服务器硬件结构，SMP（对称多处理机结构），集群，MPP（大规模并行处理），NUMA（非一致内存访问结构）

~cont.

■ SMP (Symmetric Multi-Processing)

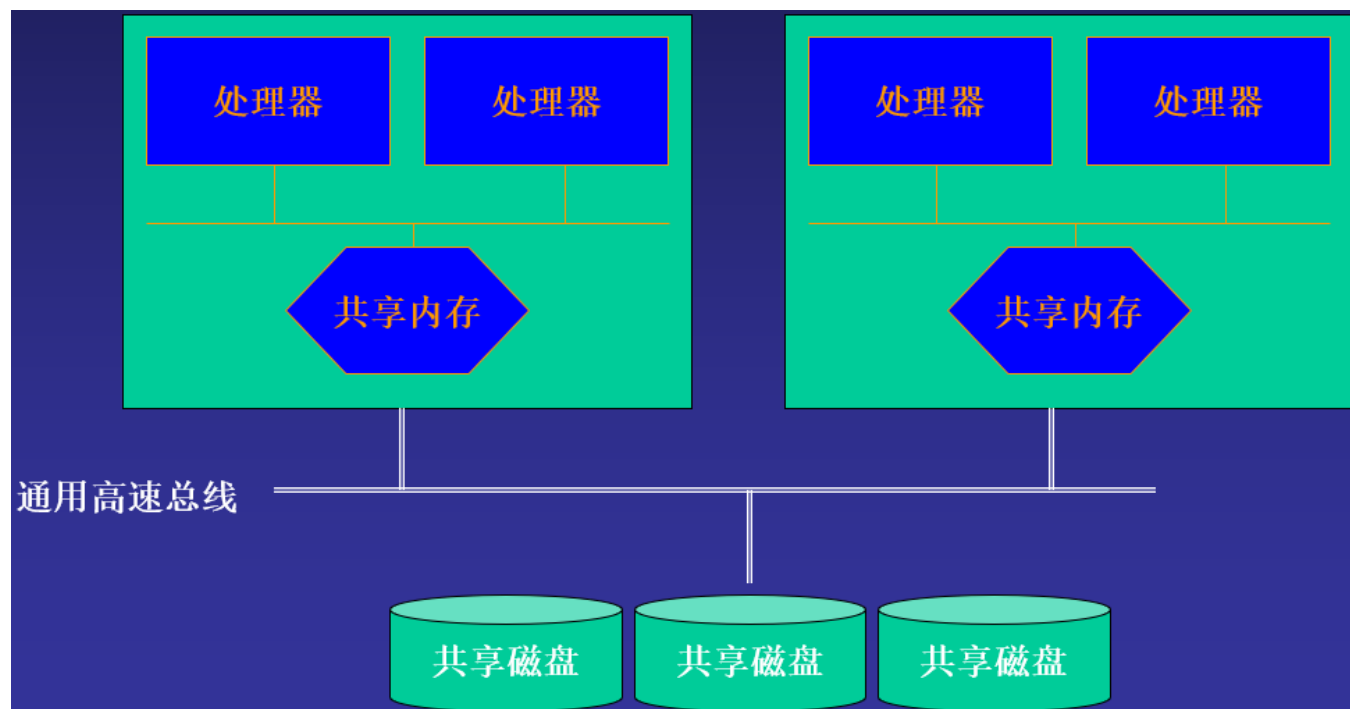
- 容易扩展，但是内存有限
- 数据规模少的情况，可以考虑采用这种结构
- 有时也被称为一致存储访问（UMA）结构体系
- 一致性意指无论在什么时候，处理器只能为内存的每个数据保持或共享唯一一个数值



~cont.

■ 集群

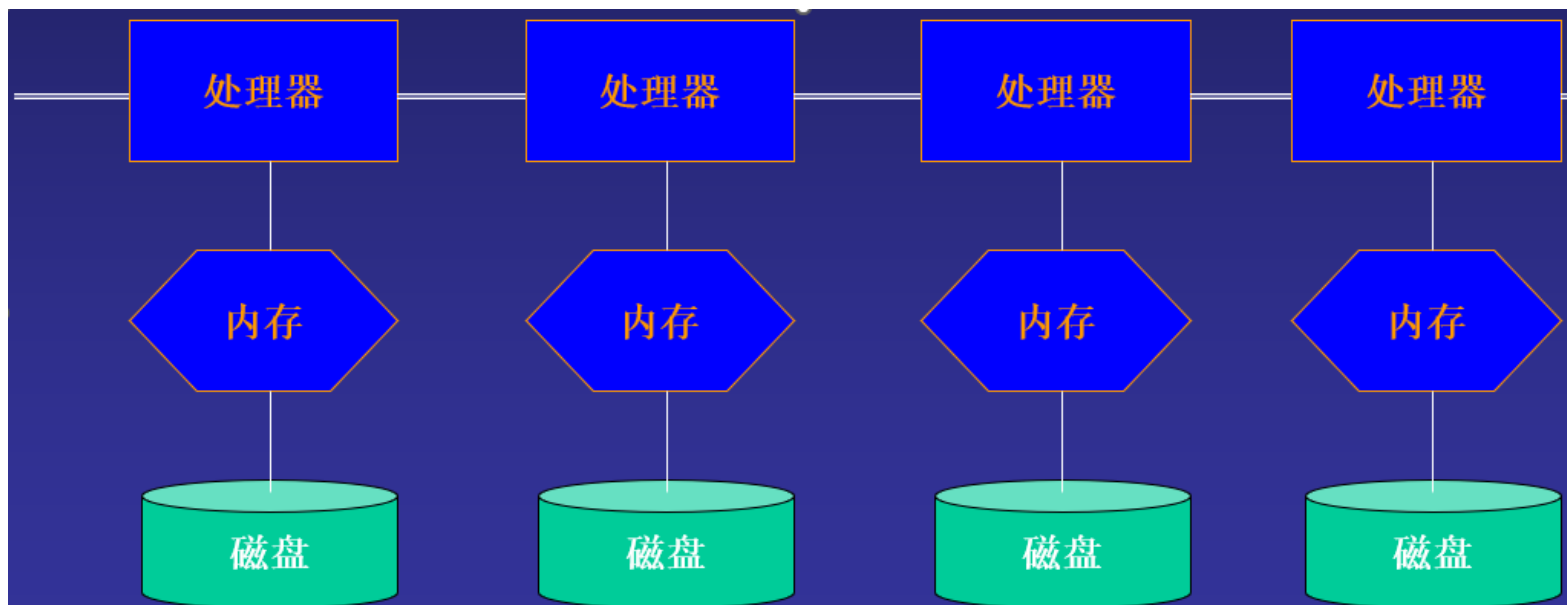
- 适用于规模不断增长的情况，容易扩展，总线带宽会限制系统扩展性
- 如果能够很好地定义数据仓库的增长，可以考虑采用这种结构



~cont.

■ MPP (Massive Parallel Processing)

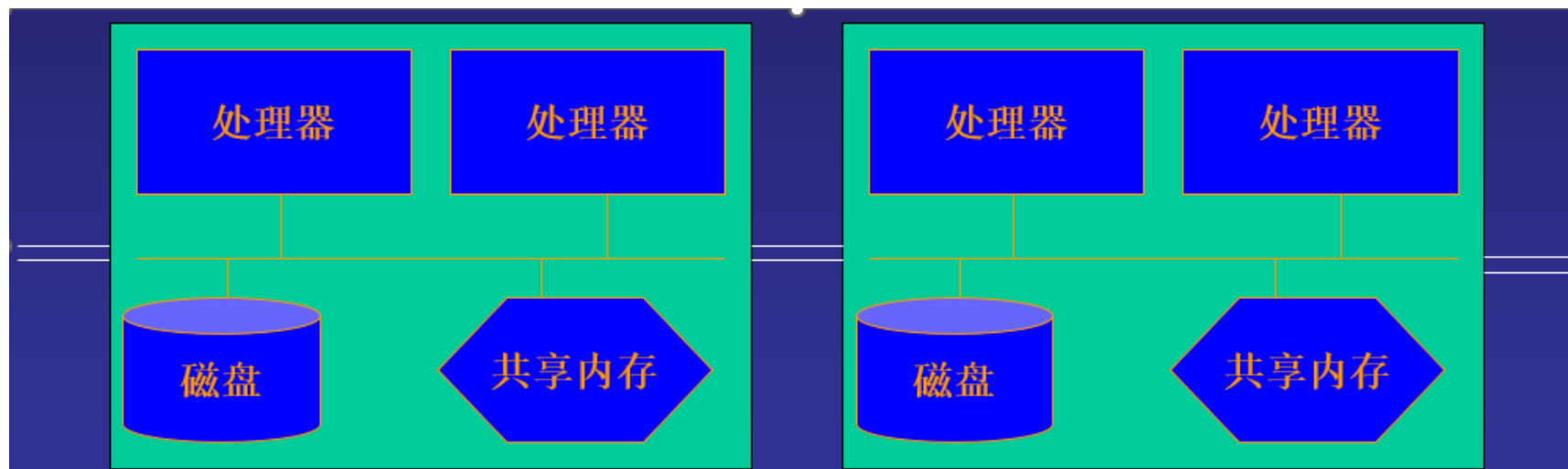
- 可高度扩展，每个节点的成本较低，失败仅限于节点之内，但是需要严格的数据划分，数据存取受到限制
- 中等数据规模, 可以考虑采用这种结构



~cont.

■ NUMA (Non-Uniform Memory Access)

- 不同的是所有节点中的处理器都可以访问全部的系统物理存储器
- 既保持了SMP模式单一操作系统拷贝、简便的应用程序编程模式以及易于管理的特点，又继承了MPP模式的可扩充性，可以有效地扩充系统的规模



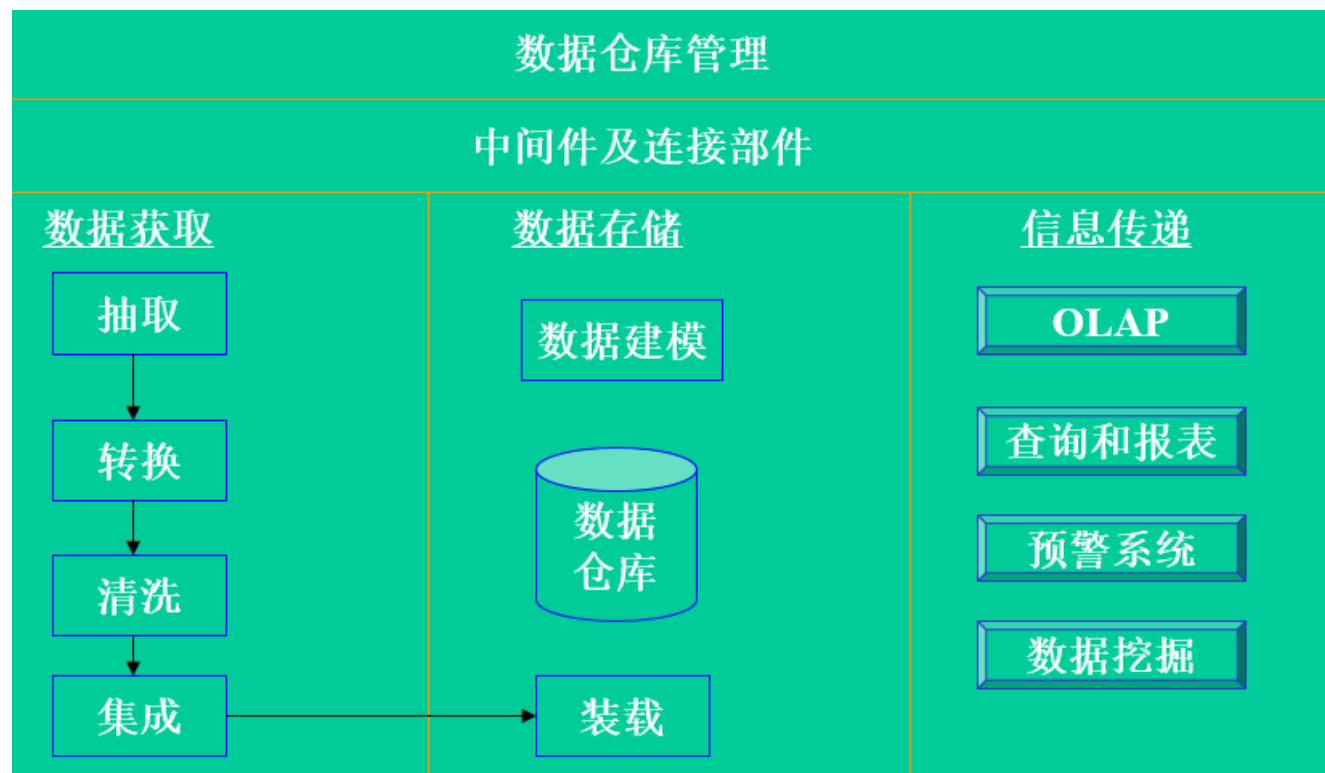
~cont.

- 计算机平台
 - 硬件和操作系统
 - 服务器硬件方案
- 数据库管理系统：DBMS选型时要保证所选的数据库管理系统与服务器硬件相匹配，除此之外还要考虑以下因素：
 - **查询优化器**。对用户的查询进行语法分析并优化
 - 查询管理。平衡不同类型查询的执行
 - 装载工具。高性能的数据装载、恢复和重启
 - 元数据管理。一个活跃的数据目录或数据字典
 - 规模的可扩展性。用户数量和数据容量都可扩展
 - 功能扩展能力。能够有多种扩展到OLAP数据库的方式
 - 查询工具应用编程接口。对领先厂商的工具开放
 - 管理功能。支持所有的数据库管理功能

~cont.

■ 工具：

– 在数据仓库的开发周期中的不同阶段都会使用到不同的工具，如图所示



7.4应用分析

- 数据仓库应用

- OLAP模型设计

- 思想是先分析问题中可能涉及的所有维度，针对每一个主题确定其需要的维度和度量变量，然后为每一个主题定义OLAP模型
 - 总体维度分析
 - 主题维度分析
 - 确定事实表度量变量和数据粒度
 - 定义OLAP模型，MOLAP模型采用星型，ROLAP采用雪花型

- 数据挖掘模型设计

- 信息传递

~cont.

- 数据仓库应用
 - OLAP模型设计
 - 数据挖掘模型设计
 - 根据应用范围，用户最终目标，数据的类型选择数据挖掘方法（统计分析、机器学习、模式识别和人工神经网络等），确定合适的模型，并将其参数校正到最优值
 - 信息传递

~cont.

- 数据仓库应用
 - OLAP模型设计
 - 数据挖掘模型设计
 - 信息传递
 - 用户与数据仓库交互通过6个阶段来实现
 - 根据数据仓库中的数据来定义需求
 - 从数据仓库中选择合适的子集
 - 用计算等操作来丰富选中的子集
 - 将行业含义和选择的数据联系起来
 - 将结果构建成适合用户的格式
 - 用各种方法来表示结构化的结果

7.5数据仓库部署

■ 数据仓库部署的内容

— 用户认可

- 完成所有用户界面及系统性能方面所有项目的最后测试，让用户认可，没有认可前不可强行部署。

— 初始装载

- 运行接口程序，将数据装入到数据仓库中。数据管理员要对装载的数据进行数据质量评估，以确保数据的可靠性，同时还要考虑数据的安全问题，对数据进行加密、建立索引等处理
- 装载最好从一个主题开始，先进行校验，反馈给设计者，设计者继续进行改进、完善，最后将数据完全载入

— 桌面准备

- 为用户安装桌面客户端工具，让他们能够看到相关的有价值的信息，并进行测试

— 初始培训

- 对用户进行数据仓库相关概念、内容和数据访问工具培训，让他们知道如何使用数据仓库应用，并能够了解数据仓库应用的价值

7.6数据仓库的运维

- 系统维护：是在系统已经交付使用之后为了改正错误和为了满足新的需要而修改系统的过程
- 数据仓库中数据的日常管理工作：
 - 清除过时的、不再使用的数据
 - 定期从源数据中提取数据，刷新数据仓库中的数据
 - 管理元数据等
- 数据仓库系统的完善工作：
 - **改正性维护**：在数据仓库设计与开发过程中，虽然已经进行了严格的测试，但对于一个大型的系统可能还潜藏着一些错误，改正性维护就是为了发现和改正这些错误而进行的过程
 - **适应性维护**：数据仓库的构建是基于当时的技术条件的，由于计算机科学技术发展十分迅速，每隔一定的周期硬件设备和系统软件都会发生重大的变革，适应性维护就是为了与变化了的环境相配合而进行的对系统进行修改的活动
 - **完善性维护**：因为数据仓库系统一般是采用快速原型法开发的，需要在系统的运行过程中不断地听取用户对已有功能的修改建议和增加新功能的需求，使系统日趋完善，这项工作称为完善性维护
 - **预防性维护**：预防性维护则是指为了改进系统未来的性能和功能打下基础而进行的修改工作

本章小结

- 数据仓库设计方法概述
- 数据仓库的三级模型
- 数据仓库概念模型
- 数据仓库逻辑模型
- 数据仓库物理模型
- 数据仓库设计的步骤
- 数据仓库的建设过程

作业

- 1.比较数据库设计与数仓设计的不同。
- 2.数仓流行的三级模型分别指哪三种？各自的主要工作是什么？
- 3.什么是信息包图？它的作用是什么？
- 4.什么是维度？什么是维度的分解与合成？如何对维度进行分解和合成？
- 5.什么是事实表？什么是度量和度量值？什么是多维数据集？
- 6.什么是星型模型？请给出实例说明其特点。
- 7.什么是雪花型模型？请给出实例说明其特点。
- 8.在数仓逻辑模型设计过程中如何进行粒度的选择？